

Софийски университет „Св. Кл. Охридски”

Факултет по математика и информатика

Бакалавърска програма
„Софтуерно инженерство”



Предмет: XML технологии за семантичен Уеб

Зимен семестър, 2025/2026 год.

Тема №54 „Шаблон за оформление на документация на курсов проект за учебни игри“

Курсов проект

Автори:

Виктор Карталов, 1MI0600352

Борил Василев, 0MI0600369

януари, 2026 г.

София

Съдържание

1 Въведение....3

2 Анализ на решението....3

2.1 Игрови процес....3

2.2 Семантика на входното и изходното текстово съдържание....4

2.3 Тип и представяне на аудио-визуалното съдържание....5

3 Дизайн....14

3.1 Структура на входното и изходното текстово съдържание....14

3.2 Тип и представяне на аудио-визуалното съдържание....20

3.3 Описание на проекта в Unity....24

3.4 Дизайн на нивата на играта....53

4 Използване на изкуствен интелект....54

5 Тестване....58

6 Заключение и възможно бъдещо развитие....58

7 Разпределение на работата....59

1 Въведение

Този курсов проект има за цел проектиране и разработване на учебна триизмерна видео игра в средата Unity, предназначена за подпомагане на образованието по математика, и по-специално за усвояване на знания, свързани с геометрия и стереометрични тела. В условията на съвременното образование използването на интерактивни дигитални средства и иновативни подходи под формата на игри представлява ефективен метод за повишаване на мотивацията и ангажираността на учениците.

Разработената видео игра е насочена към ученици и студенти, изучаващи основни и разширени понятия за триизмерни геометрични тела, като усвояваните знания са на ниво от 10 клас до университетска геометрия. Основната учебна цел на играта е чрез интерактивно взаимодействие с обекти в 3D среда да се подпомогне разпознаването и разбирането на свойствата на различни стереометрични тела, като това се осъществява чрез сортирането им по общ признаци.

Документът е структуриран, както следва:

- в точка 2 е направен анализ на игровия процес
- в точка 3 е описан дизайнът на играта и използването на XML за настройките ѝ
- в точка 4 е разгледано използването на изкуствен интелект
- в точка 5 е представено тестването на играта
- в точка 6 са направени заключения и предложения за бъдещо развитие
- в точка 7 е описано разпределението на работата в екипа

2 Анализ на решението

2.1 Игрови процес

Игровият процес на самата игра е прост и лесно смилаем за всякаква аудитория: играчът е в стая с различни предмети, сред които и 3D модели на стереометрични тела. В стаята се намира и шкаф с имената на категориите тела, които играчът трябва да сортира. Телата и предметите се избират случайно от различни групи (разделени по категориите на стереометричните тела и големината на предметите). За да победи, играчът трябва да събира стереометричните тела, скрити от големи и малки предмети, като трябва да премества предметите, за да открива телата зад тях и да ги събира в инвентара си. Ако мишката е насочена към тяло, то се огражда с контур. Щом телата са в инвентара той може да избира конкретно тяло и да го поставя където иска, като силует на тялото се прожектира на мястото, на което то би било поставено. Играчът трябва да постави тялото на правилния рафт на шкафа, който отговаря на групата, към която принадлежи, и ако го постави на правилния рафт, то той получава точки, а ако сгреши, точките, които ще получи при поставяне на правилното място, биха били по-малко. По този начин се създава система за

оценяване на извършеното от играча, без да го наказва твърде сурово за грешките му, по който начин се избягва загубата на мотивация при твърде много грешки, което би било по-вероятно, ако за грешен отговор например се отнемат точки. Инвентарът на играча представлява меню, което може да се отваря и затваря, в което се разполагат телата, репрезентирани от 2D изображения, след като играчът ги събере. Част от инвентара е и лентата за бърз достъп, която е винаги видима на екрана. Инвентара и лентата за бърз достъп имат слотове, на които седят телата, и менюто за бърз достъп има един активен слот - тоест тялото, което играчът държи в ръката си и може да постави. При поставяне на всички тела на правилните места, нивото приключва и играчът е изпратен в менюто за край на играта.

След приключване на играта резултатът на играча за нивото се запазва във файл, който може да бъде използван за анализ на резултатите на отделните играчи в учебна среда, с който да се оценяват резултатите и ефективността на обучението, проведено с играта, както и начин обучаваните лица да могат да сравняват резултатите си и по този начин да се мотивират да увеличават собствените си резултати.

Играта се състои от стартово меню, в което играчът може да започне игра, да напусне и да провери контролите на играта и как се играе. Щом играчът избере да започне игра трябва да въведе името си, което не може да е празно или да съдържа запетая, но няма нужда то да е уникално. Ако името е заето играчът бива предупреден, но не и спрян да продължи. След като името е въведено се отваря меню с избор на ниво, което по начало е само едно. Щом нивото е успешно завършено от играча, се отключва друго ниво, като играта в момента има 4 на брой нива. Всяко следващо ниво е с по-висока трудност от предхождащите го и трябва да се отключи, като се премине миналото ниво.

Контролите на играта съчетават мишка и клавиатура:

- За оглеждане в триизмерното пространство се използва мишка/тракпад;
- За движение в триизмерното пространство се използват бутоните WASD или стрелките на клавиатурата;
- За преместване (дърпане) на предмет или тяло се използва левият бутон на мишката;
- За събиране на тяло от земята се използва клавиша E;
- За поставяне на тяло на посоченото място се използва десният бутон на мишката;
- За избиране на елемент в лентата за бърз достъп се използва скрол колелцето на мишката;
- За отваряне и затваряне на менюто с инвентара се използва клавишът Tab.

Движението на различните предмети и използването на физически двигател прави играта динамична и ангажираща вниманието, като липсата на статика в процеса на игра може да се отрази положително на образователните цели на играта.

2.2 Семантика на входното и изходното текстово съдържание

Ключова характеристика на играта е начина, по-който могат да се зададат настройки за учебните сценарии в нея чрез XML файлове, от които зависят предметите и телата, които се използват в играта, разположенията им, условията за отключване на нивата, какво означава взаимодействието на играча с предметите и как се оценява представянето на играча.

Входните данни представляват 4 XML файла, намиращи се в папка User/AppDate/LocalLow/ihatecsharp/levels. Всеки файл представлява XML описание на игровите елементи, намиращи се в стаята и образуващи визуалната структура на играта, както и поведението на елементите в нея. Файловете следват един и същ шаблон, като той е описан по-долу в документацията.

Всеки файл задава категории стереометрични тела в различни групи, които играчът трябва да разпознава и сортира. В единия сценарий обектите имат значение на изпъкнали и вдлъбнати тела, в другия сценарий обектите се интерпретират като ръбести и валчести, в третия са призми, антипризми и пирамиди, а в четвъртия са платонови тела, върхово-транзитивни и стенно-транзитивни. Всеки файл носи следната информация, която се вкарва в играта:

- категории тела, които подлежат на сортиране;
- множество тела, съответстващи на някоя от категориите;
- множества от предмети, които крият телата от играча;
- множество от точки, на които могат да се появяват телата при зареждане на играта;
- множество от точки, на които могат да се появяват предметите от определено множество при зареждане на играта;
- настройки за шкафа, на който играчите трябва да сортират предметите.

Изходното съдържание на играта се намира в папка User/AppData/LocalLow/ihatecsharp и представлява CSV файл, в която се записва информация за изминалата игра при приключване на ниво. Данните, които се пазят, са името на играча, името на нивото, резултатът на играча, както и времето, за което нивото е приключено.

2.3 Тип и представяне на аудио-визуалното съдържание

В играта външното графично съдържание се състои основно от 3D модели и спрайтове. Някои от 3D моделите са взети от безплатни Unity asset пакети, налични в Unity Asset Store, а други са авторски и направени в Blender, трети са взети от sketchfab.com. Използвани са и безплатни аудио файлове, които се възпроизвеждат по време на игра.

Използван е asset пакетът Linework Lite, за да осигури визуално очертаване на телата при насочване на курсора към тях.

- Файлове, описващи 3D моделите на стереометричните тела:

Name	Date modified	Type	Size
bicone.fbx	15/02/2026 12:07	3D Object	16 KB
bicone.fbx.meta	15/02/2026 12:05	META File	4 KB
canonical_3_bipyramid.fbx	15/02/2026 12:07	3D Object	15 KB
canonical_3_bipyramid.fbx.meta	15/02/2026 12:05	META File	4 KB
canonical_4_pyramid.fbx	15/02/2026 12:07	3D Object	15 KB
canonical_4_pyramid.fbx.meta	15/02/2026 12:05	META File	4 KB
canonical_4_trapezohedron.fbx	15/02/2026 12:07	3D Object	15 KB
canonical_4_trapezohedron.fbx.meta	15/02/2026 12:05	META File	4 KB
canonical_5_bipyramid.fbx	15/02/2026 12:07	3D Object	15 KB
canonical_5_bipyramid.fbx.meta	15/02/2026 12:05	META File	4 KB
canonical_5_pyramid.fbx	15/02/2026 12:07	3D Object	16 KB
canonical_5_pyramid.fbx.meta	15/02/2026 12:05	META File	4 KB
canonical_5_trapezohedron.fbx	15/02/2026 12:07	3D Object	15 KB
canonical_5_trapezohedron.fbx.meta	15/02/2026 12:05	META File	4 KB
canonical_6_bipyramid.fbx	15/02/2026 12:07	3D Object	15 KB
canonical_6_bipyramid.fbx.meta	15/02/2026 12:05	META File	4 KB
canonical_6_pyramid.fbx	15/02/2026 12:07	3D Object	15 KB
canonical_6_pyramid.fbx.meta	15/02/2026 12:05	META File	4 KB
canonical_6_trapezohedron.fbx	15/02/2026 12:07	3D Object	15 KB
canonical_6_trapezohedron.fbx.meta	15/02/2026 12:05	META File	4 KB
cone.fbx	15/02/2026 12:07	3D Object	16 KB
cone.fbx.meta	15/02/2026 12:05	META File	4 KB
cone_biaugmented_hyperboloid.fbx	15/02/2026 12:07	3D Object	45 KB
cone_biaugmented_hyperboloid.fbx.meta	15/02/2026 12:05	META File	4 KB
cube.001.fbx	15/02/2026 12:07	3D Object	15 KB
cube.001.fbx.meta	15/02/2026 12:05	META File	4 KB
cube.fbx	15/02/2026 12:07	3D Object	15 KB
cube.fbx.meta	15/02/2026 12:05	META File	4 KB
cylinder.fbx	15/02/2026 12:07	3D Object	16 KB
cylinder.fbx.meta	15/02/2026 12:05	META File	4 KB
gyroelongated_4_bipyramid.fbx	15/02/2026 12:07	3D Object	15 KB
gyroelongated_4_bipyramid.fbx.meta	15/02/2026 12:05	META File	4 KB

Name	Date modified	Type	Size
hyperboloid.fbx	15/02/2026 12:07	3D Object	43 KB
hyperboloid.fbx.meta	15/02/2026 12:05	META File	4 KB
regular_dodecahedron.fbx	15/02/2026 12:07	3D Object	15 KB
regular_dodecahedron.fbx.meta	15/02/2026 12:05	META File	4 KB
regular_faced_4_pyramid.fbx	15/02/2026 12:07	3D Object	15 KB
regular_faced_4_pyramid.fbx.meta	15/02/2026 12:05	META File	4 KB
regular_faced_5_pyramid.fbx	15/02/2026 12:07	3D Object	15 KB
regular_faced_5_pyramid.fbx.meta	15/02/2026 12:05	META File	4 KB
regular_icosahedron.fbx	15/02/2026 12:07	3D Object	15 KB
regular_icosahedron.fbx.meta	15/02/2026 12:05	META File	4 KB
regular_octahedron.001.fbx	15/02/2026 12:07	3D Object	15 KB
regular_octahedron.001.fbx.meta	15/02/2026 12:05	META File	4 KB
regular_octahedron.fbx	15/02/2026 12:07	3D Object	15 KB
regular_octahedron.fbx.meta	15/02/2026 12:05	META File	4 KB
regular_tetrahedron.fbx	15/02/2026 12:07	3D Object	15 KB
regular_tetrahedron.fbx.meta	15/02/2026 12:05	META File	4 KB
snub_disphenoid.fbx	15/02/2026 12:07	3D Object	15 KB
snub_disphenoid.fbx.meta	15/02/2026 12:05	META File	4 KB
sphere.fbx	15/02/2026 12:07	3D Object	32 KB
sphere.fbx.meta	15/02/2026 12:05	META File	4 KB
spheroid.fbx	15/02/2026 12:07	3D Object	32 KB
spheroid.fbx.meta	15/02/2026 12:05	META File	4 KB
stellated_dodecahedron_1.fbx	15/02/2026 12:07	3D Object	16 KB
stellated_dodecahedron_1.fbx.meta	15/02/2026 12:05	META File	4 KB
stellated_dodecahedron_2.fbx	15/02/2026 12:07	3D Object	15 KB
stellated_dodecahedron_2.fbx.meta	15/02/2026 12:05	META File	4 KB
stellated_icosahedron_4.fbx	15/02/2026 12:07	3D Object	18 KB
stellated_icosahedron_4.fbx.meta	15/02/2026 12:05	META File	4 KB
stellated_octahedron_1.fbx	15/02/2026 12:07	3D Object	15 KB
stellated_octahedron_1.fbx.meta	15/02/2026 12:05	META File	4 KB
toroid.fbx	15/02/2026 12:07	3D Object	31 KB
toroid.fbx.meta	15/02/2026 12:05	META File	4 KB

torus.fbx	15/02/2026 12:07	3D Object	31 KB
torus.fbx.meta	15/02/2026 12:05	META File	4 KB
uniform_3_prism.fbx	15/02/2026 12:07	3D Object	15 KB
uniform_3_prism.fbx.meta	15/02/2026 12:05	META File	4 KB
uniform_4_antiprism.fbx	15/02/2026 12:07	3D Object	15 KB
uniform_4_antiprism.fbx.meta	15/02/2026 12:05	META File	4 KB
uniform_5_antiprism.fbx	15/02/2026 12:07	3D Object	15 KB
uniform_5_antiprism.fbx.meta	15/02/2026 12:05	META File	4 KB
uniform_5_prism.fbx	15/02/2026 12:07	3D Object	15 KB
uniform_5_prism.fbx.meta	15/02/2026 12:05	META File	4 KB
uniform_6_antiprism.fbx	15/02/2026 12:07	3D Object	15 KB
uniform_6_antiprism.fbx.meta	15/02/2026 12:05	META File	4 KB
uniform_6_prism.fbx	15/02/2026 12:07	3D Object	15 KB
uniform_6_prism.fbx.meta	15/02/2026 12:05	META File	4 KB

- Файлове, описващи 3D моделите на предметите в играта:

Name	Date modified	Type	Size
bed.fbx	15/02/2026 12:05	3D Object	33 KB
bed.fbx.meta	15/02/2026 12:05	META File	4 KB
book.001.fbx	15/02/2026 12:05	3D Object	49,499 KB
book.001.fbx.meta	15/02/2026 12:05	META File	4 KB
book.002.fbx	15/02/2026 12:05	3D Object	49,499 KB
book.002.fbx.meta	15/02/2026 12:05	META File	4 KB
book.fbx	15/02/2026 12:05	3D Object	49,499 KB
book.fbx.meta	15/02/2026 12:05	META File	4 KB
bowl.fbx	15/02/2026 12:05	3D Object	24 KB
bowl.fbx.meta	15/02/2026 12:05	META File	4 KB
cabinet.001.fbx	15/02/2026 12:05	3D Object	49,505 KB
cabinet.001.fbx.meta	15/02/2026 12:05	META File	4 KB
cabinet.fbx	15/02/2026 12:05	3D Object	49,504 KB
cabinet.fbx.meta	15/02/2026 12:05	META File	4 KB
cardboard_box.001.fbx	15/02/2026 12:05	3D Object	20 KB
cardboard_box.001.fbx.meta	15/02/2026 12:05	META File	4 KB
cardboard_box.002.fbx	15/02/2026 12:05	3D Object	20 KB
cardboard_box.002.fbx.meta	15/02/2026 12:05	META File	4 KB
cardboard_box.003.fbx	15/02/2026 12:05	3D Object	20 KB
cardboard_box.003.fbx.meta	15/02/2026 12:05	META File	4 KB
cardboard_box.004.fbx	15/02/2026 12:05	3D Object	20 KB
cardboard_box.004.fbx.meta	15/02/2026 12:05	META File	4 KB
cardboard_box.fbx	15/02/2026 12:05	3D Object	20 KB
cardboard_box.fbx.meta	15/02/2026 12:05	META File	4 KB
coffee_pot.fbx	15/02/2026 12:05	3D Object	31 KB
coffee_pot.fbx.meta	15/02/2026 12:05	META File	4 KB
cup.fbx	15/02/2026 12:05	3D Object	28 KB
cup.fbx.meta	15/02/2026 12:05	META File	4 KB
cutting_board.fbx	15/02/2026 12:05	3D Object	22 KB
cutting_board.fbx.meta	15/02/2026 12:05	META File	4 KB
cutting_board_with_handle.fbx	15/02/2026 12:05	3D Object	22 KB
cutting_board_with_handle.fbx.meta	15/02/2026 12:05	META File	4 KB

Name	Date modified	Type	Size
desk.fbx	15/02/2026 12:05	3D Object	49,502 KB
desk.fbx.meta	15/02/2026 12:05	META File	4 KB
file_binder.fbx	15/02/2026 12:06	3D Object	49,500 KB
file_binder.fbx.meta	15/02/2026 12:06	META File	4 KB
fridge.fbx	15/02/2026 12:06	3D Object	23 KB
fridge.fbx.meta	15/02/2026 12:06	META File	4 KB
kettle.fbx	15/02/2026 12:06	3D Object	31 KB
kettle.fbx.meta	15/02/2026 12:06	META File	4 KB
lamp.fbx	15/02/2026 12:06	3D Object	34 KB
lamp.fbx.meta	15/02/2026 12:06	META File	4 KB
magazine_holder.fbx	15/02/2026 12:06	3D Object	49,500 KB
magazine_holder.fbx.meta	15/02/2026 12:06	META File	4 KB
mini_fridge.fbx	15/02/2026 12:06	3D Object	22 KB
mini_fridge.fbx.meta	15/02/2026 12:06	META File	4 KB
mug.fbx	15/02/2026 12:06	3D Object	26 KB
mug.fbx.meta	15/02/2026 12:06	META File	4 KB
oven.fbx	15/02/2026 12:06	3D Object	36 KB
oven.fbx.meta	15/02/2026 12:06	META File	4 KB
painting.001.fbx	15/02/2026 12:06	3D Object	20 KB
painting.001.fbx.meta	15/02/2026 12:06	META File	4 KB
painting.fbx	15/02/2026 12:06	3D Object	20 KB
painting.fbx.meta	15/02/2026 12:06	META File	4 KB
pan.fbx	15/02/2026 12:06	3D Object	27 KB
pan.fbx.meta	15/02/2026 12:06	META File	4 KB
pillow.fbx	15/02/2026 12:06	3D Object	25 KB
pillow.fbx.meta	15/02/2026 12:06	META File	4 KB
plate.fbx	15/02/2026 12:06	3D Object	24 KB
plate.fbx.meta	15/02/2026 12:06	META File	4 KB
pot.fbx	15/02/2026 12:06	3D Object	28 KB
pot.fbx.meta	15/02/2026 12:06	META File	4 KB
saucer.fbx	15/02/2026 12:06	3D Object	24 KB
saucer.fbx.meta	15/02/2026 12:06	META File	4 KB

school_chair.fbx	15/02/2026 12:06	3D Object	23,357 KB
school_chair.fbx.meta	15/02/2026 12:06	META File	4 KB
school_desk.fbx	15/02/2026 12:06	3D Object	23,370 KB
school_desk.fbx.meta	15/02/2026 12:06	META File	4 KB
sofa.fbx	15/02/2026 12:06	3D Object	39 KB
sofa.fbx.meta	15/02/2026 12:06	META File	4 KB
stool.fbx	15/02/2026 12:06	3D Object	24 KB
stool.fbx.meta	15/02/2026 12:06	META File	4 KB
table.fbx	15/02/2026 12:06	3D Object	23 KB
table.fbx.meta	15/02/2026 12:06	META File	4 KB
telephone.fbx	15/02/2026 12:06	3D Object	37 KB
telephone.fbx.meta	15/02/2026 12:06	META File	4 KB

- Файлове, описващи 2D изображения, използвани за текстуриране на предметите:

Name	Date modified	Type	Size
board_color.png	15/02/2026 12:05	PNG File	3,105 KB
board_color.png.meta	15/02/2026 12:05	META File	4 KB
board_metallicness.png	15/02/2026 12:05	PNG File	27 KB
board_metallicness.png.meta	15/02/2026 12:05	META File	4 KB
board_roughness.png	15/02/2026 12:05	PNG File	2,042 KB
board_roughness.png.meta	15/02/2026 12:05	META File	4 KB
ceiling_color.png	15/02/2026 12:05	PNG File	2,816 KB
ceiling_color.png.meta	15/02/2026 12:05	META File	4 KB
ceiling_emission.png	15/02/2026 12:05	PNG File	41 KB
ceiling_emission.png.meta	15/02/2026 12:05	META File	4 KB
ceiling_normal_opengl.png	15/02/2026 12:05	PNG File	16,390 KB
ceiling_normal_opengl.png.meta	15/02/2026 12:05	META File	4 KB
ceiling_roughness.png	15/02/2026 12:05	PNG File	9 KB
ceiling_roughness.png.meta	15/02/2026 12:05	META File	4 KB
desk_color.png	15/02/2026 12:05	PNG File	5,403 KB
desk_color.png.meta	15/02/2026 12:05	META File	4 KB
desk_metallicness.png	15/02/2026 12:05	PNG File	17 KB
desk_metallicness.png.meta	15/02/2026 12:05	META File	4 KB
desk_normal_opengl.png	15/02/2026 12:05	PNG File	15,701 KB
desk_normal_opengl.png.meta	15/02/2026 12:05	META File	4 KB
desk_roughness.png	15/02/2026 12:05	PNG File	2,201 KB
desk_roughness.png.meta	15/02/2026 12:05	META File	4 KB
door_color.png	15/02/2026 12:05	PNG File	5,719 KB
door_color.png.meta	15/02/2026 12:05	META File	4 KB
door_metallicness.png	15/02/2026 12:05	PNG File	13 KB
door_metallicness.png.meta	15/02/2026 12:05	META File	4 KB
door_normal_opengl.png	15/02/2026 12:05	PNG File	15,748 KB
door_normal_opengl.png.meta	15/02/2026 12:05	META File	4 KB
door_roughness.png	15/02/2026 12:05	PNG File	1,620 KB
door_roughness.png.meta	15/02/2026 12:05	META File	4 KB
parquet_color.png	15/02/2026 12:05	PNG File	5,410 KB
parquet_color.png.meta	15/02/2026 12:05	META File	4 KB

parquet_normal_opengl.png	15/02/2026 12:05	PNG File	4,688 KB
parquet_normal_opengl.png.meta	15/02/2026 12:05	META File	4 KB
parquet_roughness.png	15/02/2026 12:05	PNG File	2,372 KB
parquet_roughness.png.meta	15/02/2026 12:05	META File	4 KB
texture_color.png	15/02/2026 12:05	PNG File	25,840 KB
texture_color.png.meta	15/02/2026 12:05	META File	4 KB
texture_normal.png	15/02/2026 12:05	PNG File	23,640 KB
texture_normal.png.meta	15/02/2026 12:05	META File	4 KB
wall_color.png	15/02/2026 12:05	PNG File	5,245 KB
wall_color.png.meta	15/02/2026 12:05	META File	4 KB
wall_normal_opengl.png	15/02/2026 12:05	PNG File	22,875 KB
wall_normal_opengl.png.meta	15/02/2026 12:05	META File	4 KB
wall_roughness.png	15/02/2026 12:05	PNG File	2,645 KB
wall_roughness.png.meta	15/02/2026 12:05	META File	4 KB
window_color.png	15/02/2026 12:05	PNG File	5,317 KB
window_color.png.meta	15/02/2026 12:05	META File	4 KB
window_metallicness.png	15/02/2026 12:05	PNG File	91 KB
window_metallicness.png.meta	15/02/2026 12:05	META File	4 KB
window_normal_opengl.png	15/02/2026 12:05	PNG File	14,469 KB
window_normal_opengl.png.meta	15/02/2026 12:05	META File	4 KB
window_opacity.png	15/02/2026 12:05	PNG File	68 KB
window_opacity.png.meta	15/02/2026 12:05	META File	4 KB
window_roughness.png	15/02/2026 12:05	PNG File	2,343 KB
window_roughness.png.meta	15/02/2026 12:05	META File	4 KB

- Файлове, описващи изображения, нужни за UI елементите:

border.png	Type: PNG File Dimensions: 128 x 128	Size: 617 bytes
border.png.meta Type: META File		Date modified: 15/02/2026 12:05 Size: 3.09 KB
slot.png	Type: PNG File Dimensions: 128 x 128	Size: 524 bytes
slot.png.meta Type: META File		Date modified: 15/02/2026 12:05 Size: 3.09 KB

- Аудио файлове:

Name	Date modified	Type	Size
buzz.wav	15/02/2026 12:05	WAV File	44 KB
buzz.wav.meta	15/02/2026 12:05	META File	1 KB
mallet.wav	15/02/2026 12:05	WAV File	81 KB
mallet.wav.meta	15/02/2026 12:05	META File	1 KB
ring.wav	15/02/2026 12:05	WAV File	104 KB
ring.wav.meta	15/02/2026 12:05	META File	1 KB
rings.wav	15/02/2026 12:05	WAV File	259 KB
rings.wav.meta	15/02/2026 12:05	META File	1 KB
thump.wav	15/02/2026 12:05	WAV File	156 KB
thump.wav.meta	15/02/2026 12:05	META File	1 KB
win.wav	15/02/2026 12:05	WAV File	744 KB
win.wav.meta	15/02/2026 12:05	META File	1 KB

- Шрифтови файлове:

Name	Date modified	Type	Size
ocr_a.ttf	15/02/2026 12:05	TrueType font file	16 KB
ocr_a.ttf.meta	15/02/2026 12:05	META File	1 KB
ocr_a_sdf.asset	15/02/2026 12:05	ASSET File	568 KB
ocr_a_sdf.asset.meta	15/02/2026 12:05	META File	1 KB

- Иконите на телата трябва да се съхраняват в директория AppData/LocalLow/ihatecsharp/images и общо заемат 1.4 мегабайта пространство.

3 Дизайн

3.1 Структура на входното и изходното текстово съдържание

XML файловете се използват, като от тях се зарежда информацията за предметите и телата в играта, позицията на точките, на които могат да се появят при генериране на ниво, както и информация за категориите, на които се разделят телата в нивото. Един от шаблонните файлове, които описват ниво в играта, изглежда по този начин:

```
<configuration>
  <categories>
    <category id="0" score="10" max_spawn_count="5"/>
    <category id="1" score="16" max_spawn_count="8"/>
  </categories>
  <item_pool>
  ...
  </item_pool>
  <prop_pools>
  ...
  </prop_pools>
  <prop_spawn_points>
  ...
  </prop_spawn_points>
  <item_spawn_points>
  ...
  </item_spawn_points>
  <props>
  ...
  </props>
</configuration>
```

Елементът categories съдържа поделементите category, които имат атрибути id, score и max_spawn count. Id-то е идентификационния номер, с който се свързва дадена категория (в случая 0 е многостен, а 1 е ротационно тяло)

Елементът item_pool представлява елемент, който съдържа информация за всички тела, които могат да се появят в описаното ниво. В item_pool се съдържат елементи item, които изглеждат по следния начин:

```
<!-- polyhedra -->
<item weight="1" category_id="0">
  <prefab name="regular_tetrahedron"/>
  <image path="regular_tetrahedron.png"/>
  <description>regular tetrahedron</description>
  <mass>1</mass>
</item>
<item weight="1" category_id="0">
  <prefab name="cube"/>
  <image path="cube"/>
  <description>cube</description>
  <mass>1</mass>
</item>
<item weight="1" category_id="0">
  <prefab name="regular_octahedron"/>
  <image path="regular_octahedron"/>
  <description>regular octahedron</description>
  <mass>1</mass>
</item>
```

Всеки item (тяло) има атрибути weight (тегло) - честота на появяване и category_id (id на категория, към която принадлежи). Има поделементи prefab с атрибут name (име на игровия обект в Unity, който представлява съответното тяло), image с атрибут path (път към 2D изображението на предмета, което се използва в инвентара), description (това което се визуализира в играта, щом курсора на мишката сочи към тялото в инвентара на играча) и mass (маса на тялото за физическия двигател на Unity).

Елементът prop_pools съдържа поделементи prop_pool, които за различните групи от предмети (prop), които се появяват в сцената. Съответно всеки prop_pool съдържа поделементи prop.

```
<prop_pools>
  <!--  desks  -->
  <prop_pool id="0" empty_weight="0">
    <prop weight="2">
      <prefab name="school_desk"/>
      <mass>18</mass>
    </prop>
    <prop weight="1">
      <prefab name="school_desk.001"/>
      <mass>18</mass>
    </prop>
    <prop weight="1">
      <prefab name="school_desk.002"/>
      <mass>18</mass>
    </prop>
  </prop_pool>
  <!--  chairs  -->
  <prop_pool id="1" empty_weight="2">
  ...
  </prop_pool>
  <!--  chairs  -->
  <prop_pool weight="2" id="2" empty_weight="0">
  ...
  </prop_pool>
  <!--  cabinets  -->
  <prop_pool id="3" empty_weight="0">
  ...
  </prop_pool>
  <!--  sofa  -->
  <prop_pool id="4" empty_weight="1">
  ...
  </prop_pool>
  <!--  teacher desk  -->
  <prop_pool id="5" empty_weight="0">
  ...
  </prop_pool>
  <!--  medium props  -->
  <prop_pool id="6" empty_weight="1">
  ...
  </prop_pool>
</prop_pools>
```

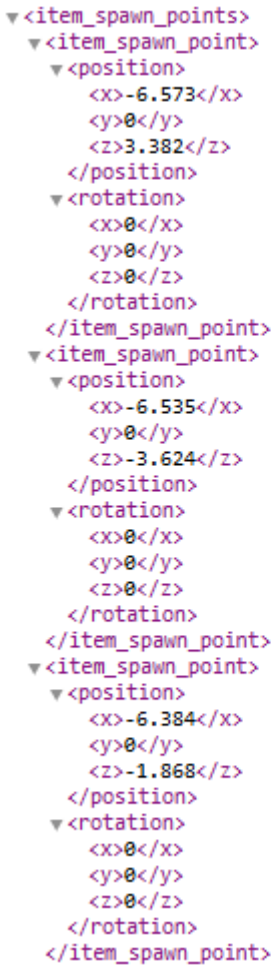
Както item_pool, prop_pool също има атрибут pool_id, но има и атрибут empty_weight (шанса да не се избере нищо при избиране на предмет).

Елементите prop са подобни на item, само че при тях липсва category_id (те не се сортират и събират), image и description (защото не могат да бъдат в инвентара на играча).

Елементът prop_spawn_points съдържа поделементи prop_spawn_point, които описват точките, на които могат да се появяват предмети. Случаен предмет от пула, който принадлежи на дадена точка, се появява на нея при зареждане на играта.

```
<prop_spawn_points>
  <!--  desks  -->
  <prop_spawn_point prop_pool_id="0">
    <position>
      <x>3.038</x>
      <y>0</y>
      <z>2.388</z>
    </position>
    <rotation>
      <x>0</x>
      <y>0</y>
      <z>0</z>
    </rotation>
  </prop_spawn_point>
  <prop_spawn_point prop_pool_id="0">
    <position>
      <x>3.08</x>
      <y>0</y>
      <z>-0.16</z>
    </position>
    <rotation>
      <x>0</x>
      <y>0</y>
      <z>0</z>
    </rotation>
  </prop_spawn_point>
  <prop_spawn_point prop_pool_id="0">
    <position>
      <x>3.28</x>
      <y>0</y>
      <z>-2.41</z>
    </position>
    <rotation>
      <x>0</x>
      <y>0</y>
      <z>0</z>
    </rotation>
  </prop_spawn_point>
```

Всеки prop_spawn_point има prop_pool_id (id на пул от който се вземат предметите), както и елементи prop. prop елементите съдържат поделементи position и rotation, а те имат поделементи x, y и z, които определят съответно позицията и ротацията на точката в триизмерното пространство.



Подобно на prop_spawn_points, item_spawn_points съдържа множество елементи item_spawn_point, които съдържат информация за всички точки за появяване на тела в играта. Те също съхраняват информация за позиция и ротация, но нямат атрибут pool_id (защото на всяка точка може да се появи тяло от който и да е pool).

```
<props>
  <prop>
    <prefab name="shelves"/>
    <position>
      <x>5.75</x>
      <y>0</y>
      <z>-2.6</z>
    </position>
    <rotation>
      <x>0</x>
      <y>205</y>
      <z>0</z>
    </rotation>
    <!--  <bin cateogry_id="1" />  -->
    <labels>
      <label>
        <text>rubesti</text>
        <horizontal_alignment>left</horizontal_alignment>
        <vertical_alignment>top</vertical_alignment>
        <position>
          <x>0.125</x>
          <y>0.825</y>
          <z>0</z>
        </position>
        <rotation>
          <x>0</x>
          <y>-90</y>
          <z>0</z>
        </rotation>
        <width>1.5</width>
        <height>0.35</height>
      </label>
      <label>
      ...
      </label>
    </labels>
    <sockets>
      <socket category_id="0">
        <position>
          <x>0</x>
          <y>0.825</y>
          <z>-0.6</z>
        </position>
        <rotation>
          <x>0</x>
          <y>0</y>
          <z>0</z>
        </rotation>
        <size>
          <x>0.3</x>
          <y>0.35</y>
          <z>0.3</z>
        </size>
      </socket>
```

Последният елемент е елементът props, който съдържа информация за страничните елементи в играта, а по-точно шкафа, на който се нареждат тела, слотовете за тела и надписите на шкафа. Поделементът prop съдържа вече разгледаните елементи position и rotation, както и prefab с артибут name, който посочва името на предефинирания игрови обект, който се използва за шкафа в играта. Поделементът на prop labels съдържа множество от елементи label, които съдържат данните за надписите на шкафа, показващи категорията, която трябва да се нареди на съответното място, те отново имат position и rotation, както и width и height, които задават дължината и височината на текстовото поле, text, който указва текстът, който трябва да се визуализира в текстовото поле, както и

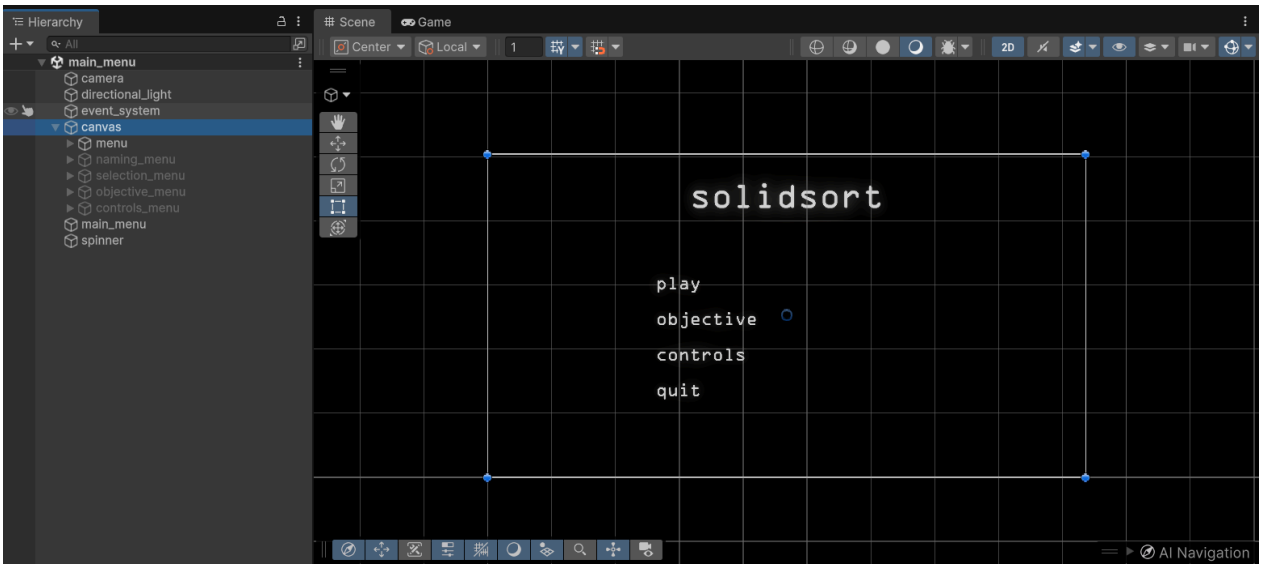
horizontal_alignment и vertical_alignment, които указват позиционирането на текста в текстовото поле. Последният поделемент на prop е sockets, съдържащ множество от socket, които описват слотовете за поставяне на тела в рафтовете на шкафа. Всеки socket има атрибут category_id, който описва id-то на категорията тела, която трябва да приема съответният слот. Поделементите на socket са отново position и rotation. Има само един елемент от тип prop (в случая шкаф), но не е изключено да се добавят още в зависимост от случая.

Изходния файл е leaderboard.csv - прост csv файл, в който данните се пазят като comma-separated values.

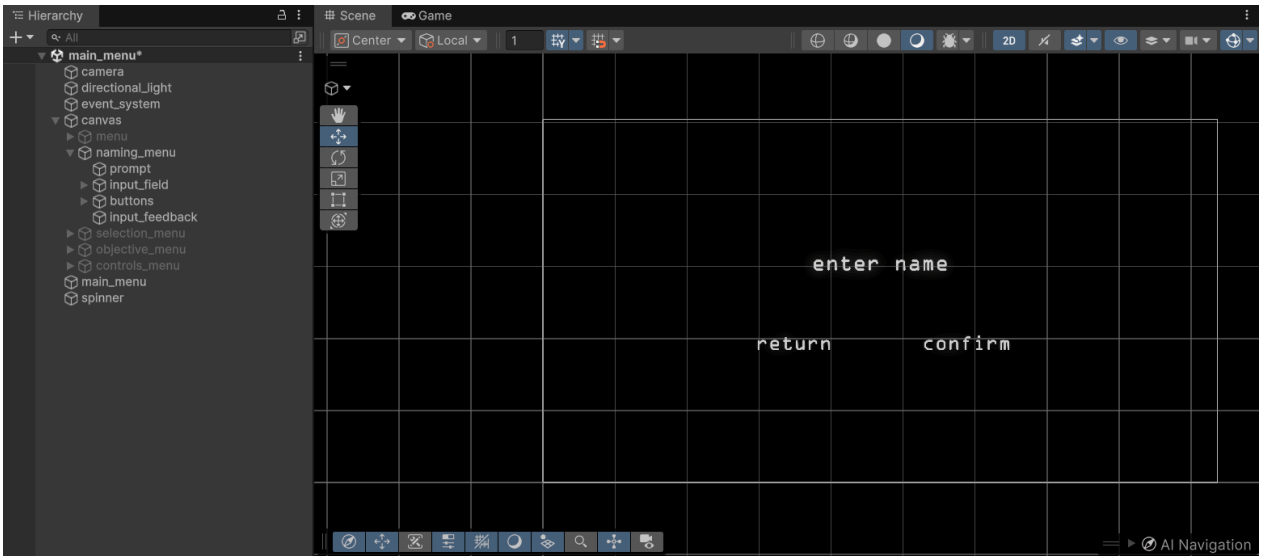
3.2 Тип и представяне на аудио-визуалното съдържание

В играта има 3 сцени: начално меню, стая и краен екран. Всеки от тях съдържа елементи, поставени в Unity редактора, които представляват структурата на играта във визуална среда.

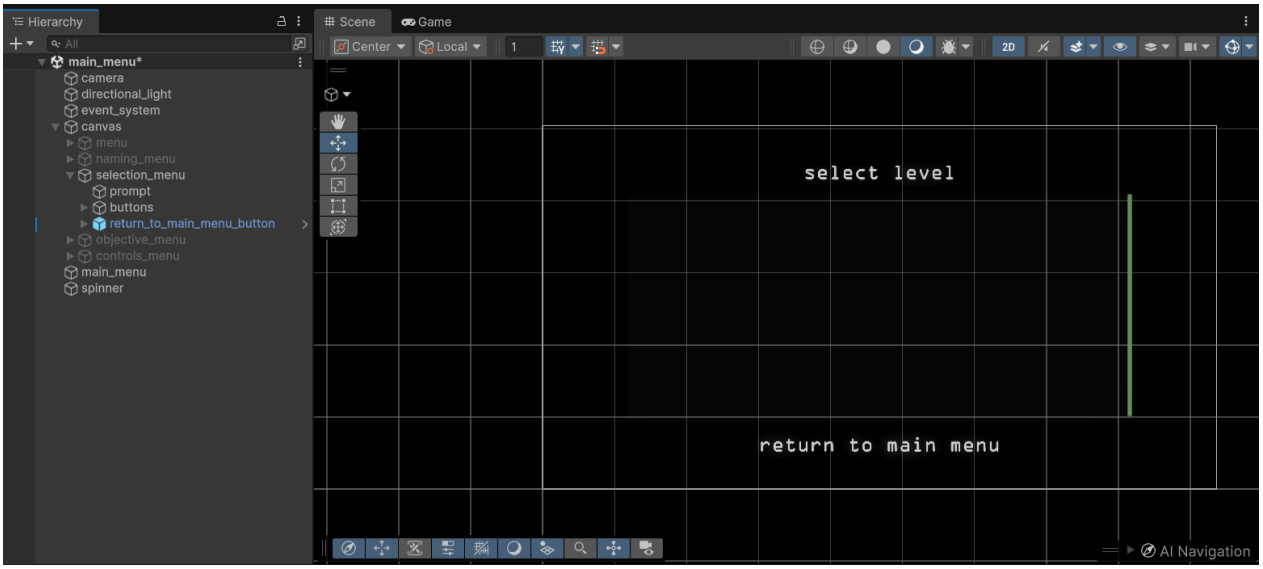
Първият екран, main_menu, съдържа 6 главни обекта, нужни за функционирането му. Обектите camera и directional_light са нужни за осветяването и визуализирането на 3D обект, който се върти във фона на играта. Този 3D обект се нарича spinner и е представен чрез обекта spinner в редактора, носещ скрипта spinner.cs. Обекта main_menu е празен обект, който съдържа като компонент скрипта main_menu.cs. Обектът event_system е нужен за subscribe-ването към евенти, с което се отличава event-базирания стил на код, използван в проекта. Елементът canvas е UI елемент от Unity, който съдържа UI елементи. Той се използва навсякъде, където има 2D потребителски интерфейс. В него се съдържат поделементите menu, който съдържа бутоните в началното меню, визуализирани на тази снимка. Всеки бутон задейства различна функция от скрипта main_menu.cs, като свързването става чрез елемента main_menu.



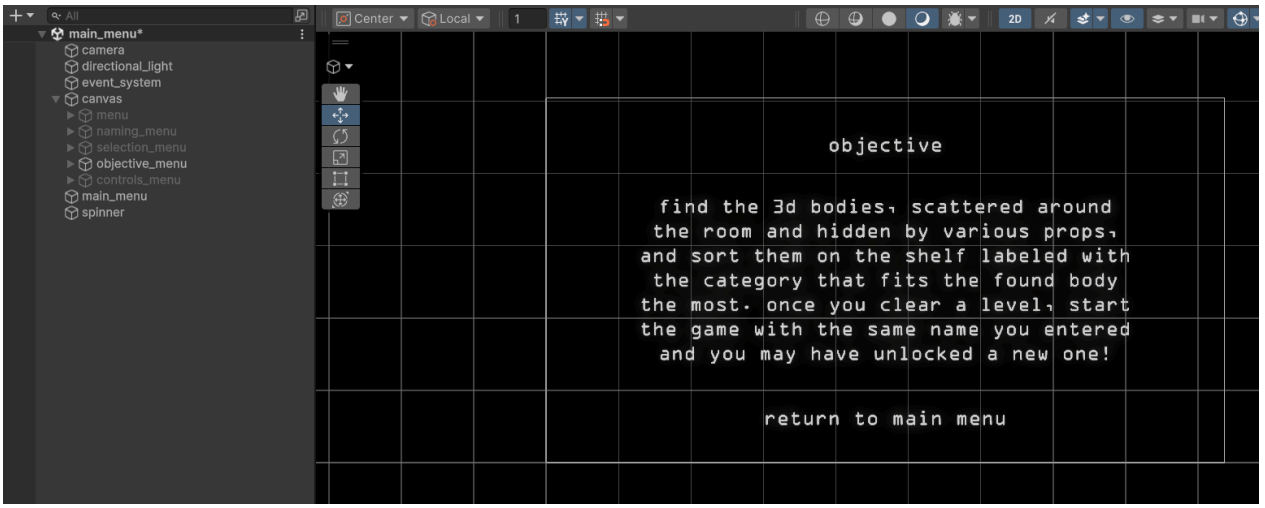
Следващото меню е naming_menu, което съдържа интерфейса, свързан с въвеждане на потребителското име на играча при започване на игра. Съдържа както бутони, така и input_field, в който се въвежда името, а и input_feedback, където се визуализират съобщения, свързани с въведеното от потребителя име.



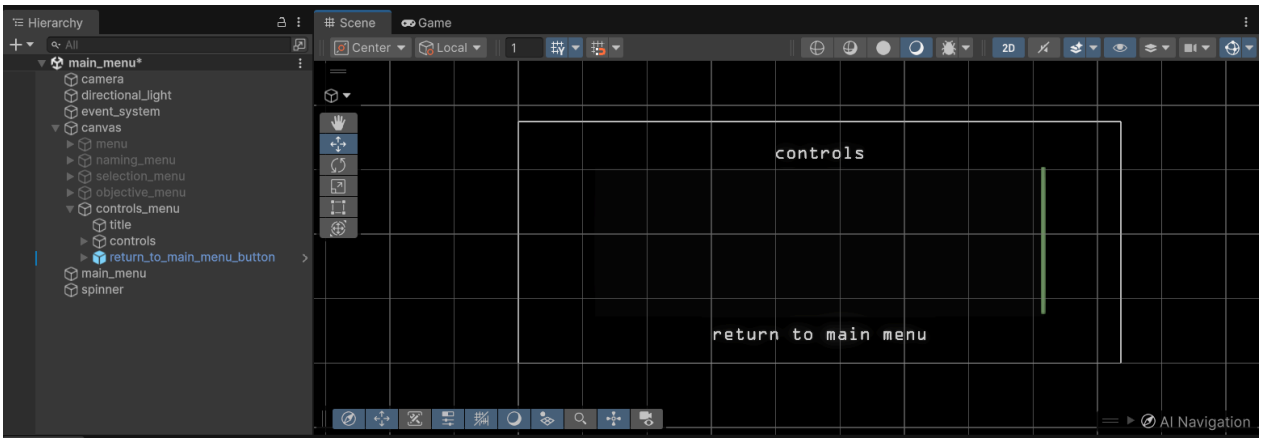
Менюто selection_menu се визуализира след въвеждането на име и представлява екран, на който играчът избира нивото, което иска да избере при текущата игра. Поделементът buttons представлява списъкът от нива, който се визуализира, като има и scrollbar.



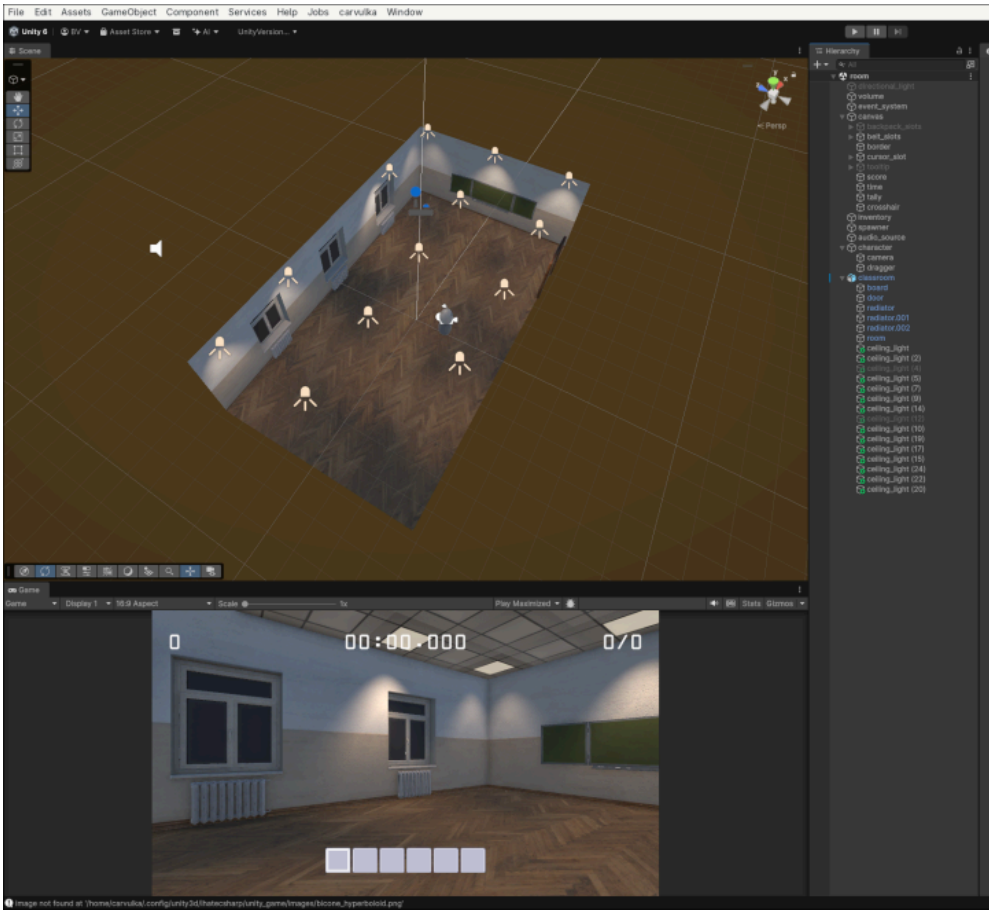
Менюто objective_menu се появява при натискане на бутон objective и представлява просто меню с текстово поле, в което е написано кратко упътване за това каква е основната цел на играта.



Менюто controls се появява при натискане на бутона controls и представлява меню, в което в елемента controls, сходен на buttons от selection_menu, от скрипта main_menu се зареждат контролите от input_system и се визуализират като елементи в кутия.



Следващата по индекс сцена е room, в която се състои същинския игрови процес в играта. В нея се намират игровите обекти volume, event_system (със същото предназначение като по-горе), canvas, inventory, spawner, audio_source, character и classroom.



Обектът volume е нужен за рендериране на по-сложни графични ефекти в 3D средата. Стандартен Unity обект е.

Обектът canvas представлява 2D интерфейса на играта, който включва лентата за бърз достъп, инвентара, таймера, брояча на точките и брояча на намерените тела. В него поделементът cursor_slot съдържа скрипта cursor_slot.cs, tooltip съдържа tooltip.cs, score съдържа score.cs, time съдържа time.cs и tally съдържа tally.cs. Обектът belt_slots съдържа множество от елементи slot, които предствляват слотовете в лентата за бърз достъп и всеки съдържа slot.cs. Подобно и backpack_slots съдържа слотове, които също съдържат този скрипт.

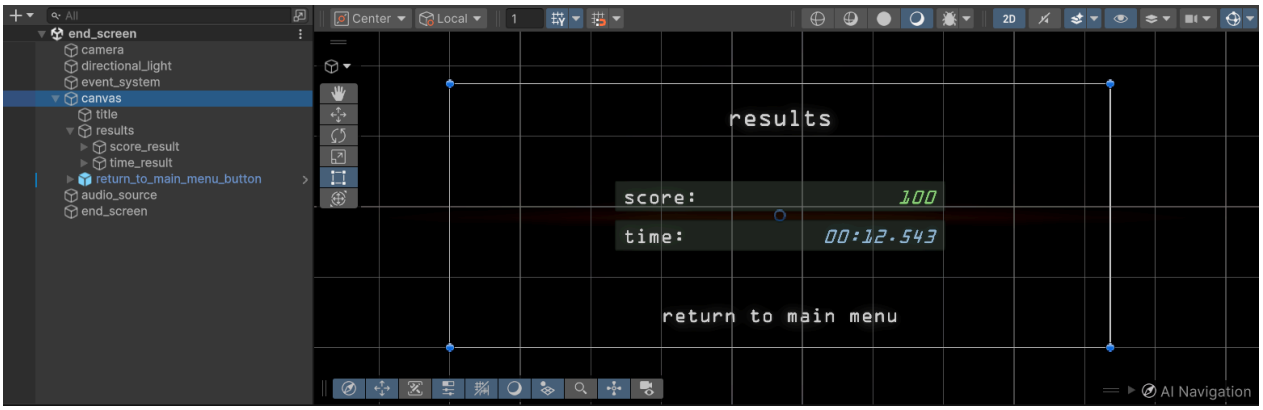
Обектът inventory е празен обект, който съдържа като компонент скрипта inventory.cs, нужен за управляване на инвентара. Обектът spawner е също празен, но съдържа скрипта spawner.cs.

Обектът audio_source е обект, от който зависи възпроизвеждането на звукови файлове по време на игра.

Обектът character представлява героя, управляван от играча. Съдържа скрипта character.cs, отговарящ за поведението на играча в играта. Поделементите му са camera, който е закачен за играча и представлява Camera обект от Unity, с който се осъществява виждането на играча в играта. Обектът dragger е нужен за местенето на предмети от играча, като щом играч тръгне да мести предмет, той се закача за този елемент и по този начин се контролира от играча.

Обектът classroom представлява визуалното пространство, в което се състои играта, като включва всички елементи на околната среда на стаята.

Последната сцена е end_screen, която показва крайният екран, който се показва при завършване на ниво. В нея има event_system, directional_light, camera и audio_source, които са описани по-горе. Освен това съдържа canvas и end_screen.



Обектът canvas е UI обекта, в който се намират всички UI елементи. В случая това са title (заглавието на менюто, текста “results”), results, включващ подобектите score_result и time_result, които визуализират резултата на играча и времето, за което се е справил с нивото, както и return_to_main_menu_button, който връща играча към началното меню.

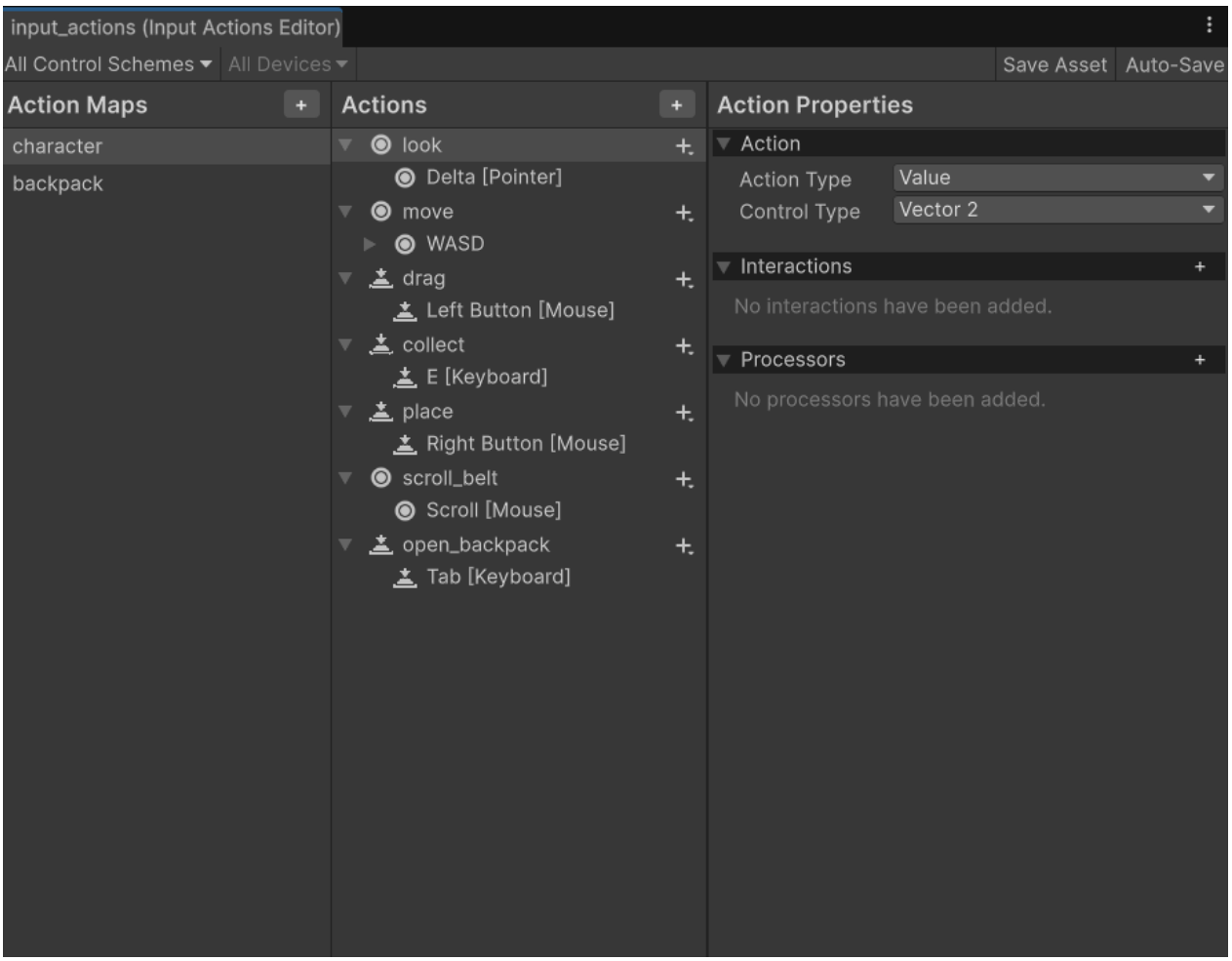
Обектът end_screen е празен обект, в който е закачен скрипта end_screen.cs, отговарящ за поведението на цялото крайно меню, като визуализирането на резултата и времето и работата на бутона.

3.3 Описание на проекта в Unity

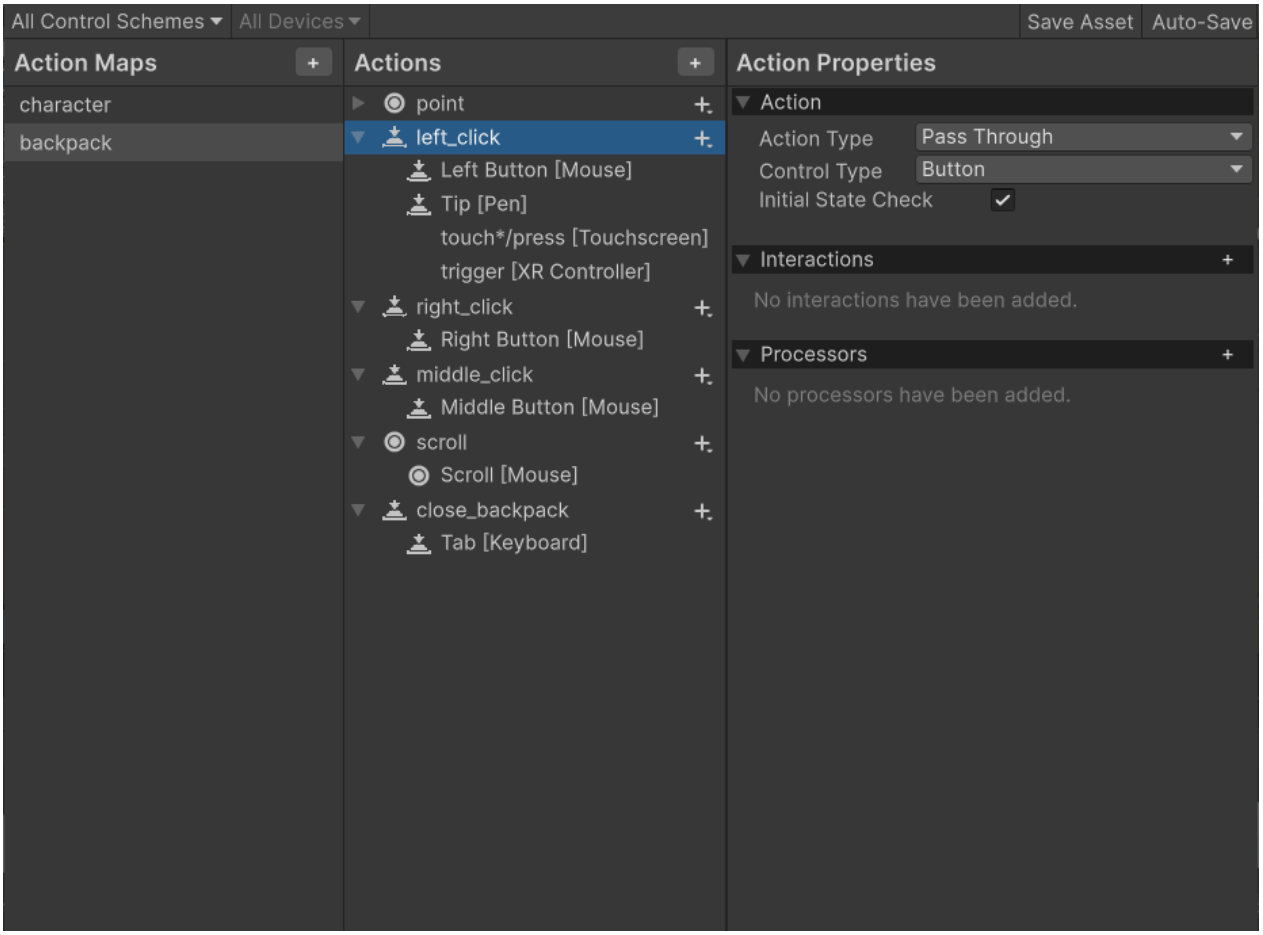
Проектът е създаден в Unity, като използва версия на редактора 6000.3.2f1, най-новата до датата на завършването на проекта. Проекта използва Universal Render Pipeline на Unity за рендериране на графиката, и е създаден като Universal 3D проект от менюто за създаване на проект на Unity.

В файла input_actions са описани входните действия, използвани за системата за вход на Unity. Има два action maps, character и backpack, които отговарят съответно за входните действия, управляващи съответно игровия обект на играча и инвентара в играта. Контролите са предвидени само за компютър, засега.

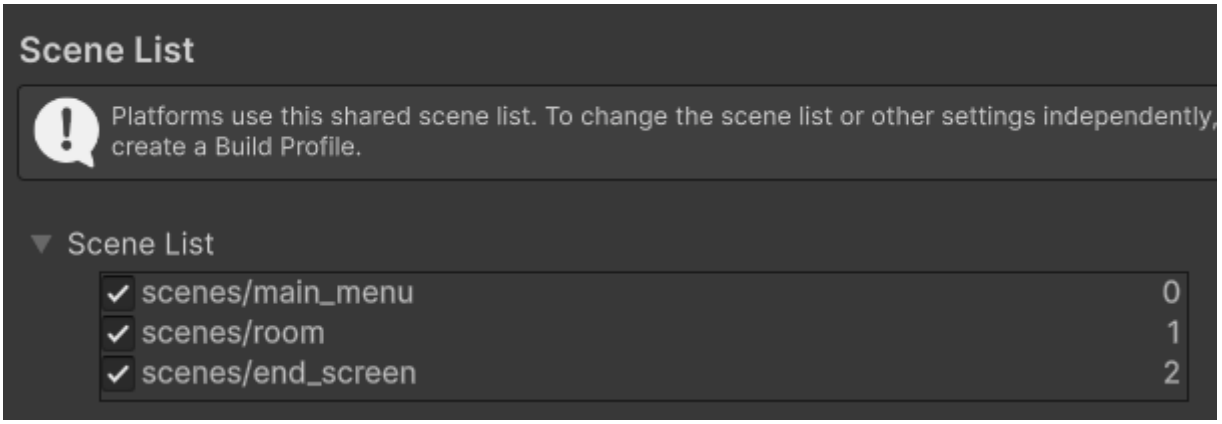
- character:


input_actions (Input Actions Editor)
All Control Schemes
All Devices
Save Asset
Auto-Save
Action Maps
character
backpack
Actions
look
Delta [Pointer]
move
WASD
drag
Left Button [Mouse]
collect
E [Keyboard]
place
Right Button [Mouse]
scroll_belt
Scroll [Mouse]
open_backpack
Tab [Keyboard]
Action Properties
Action
Action Type
Value
Control Type
Vector 2
Interactions
No interactions have been added.
Processors
No processors have been added.

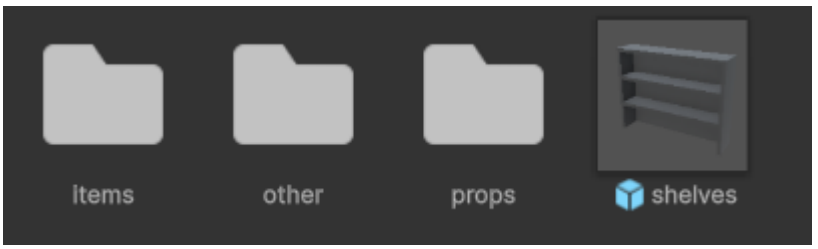
- backpack:



В build profiles на проекта са описани индексите на различните сцени в играта, които се използват при работа със SceneManager на Unity. Индексите са наредени в реда на показване на сцените по време на игра:

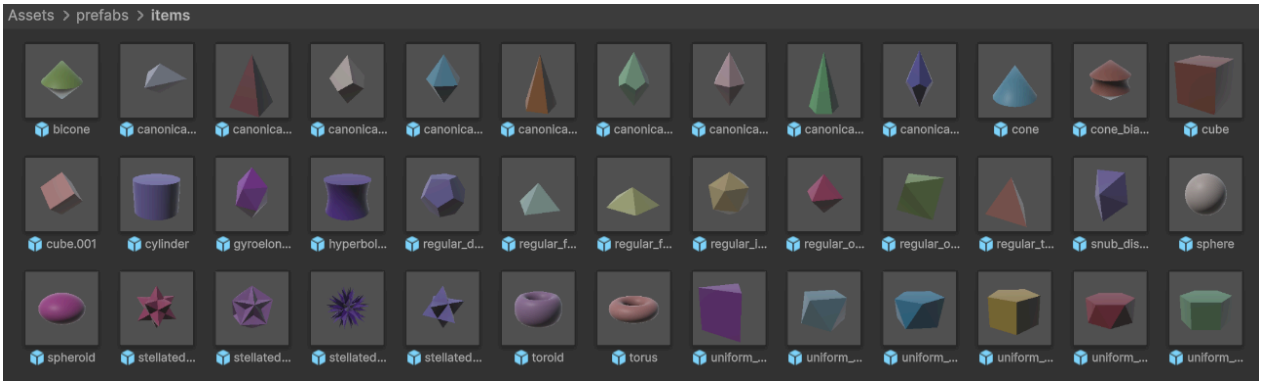


Предефинираните обекти (префабовете), използвани в играта се намират в папка prefabs, като в нея има 3 поддиректории.

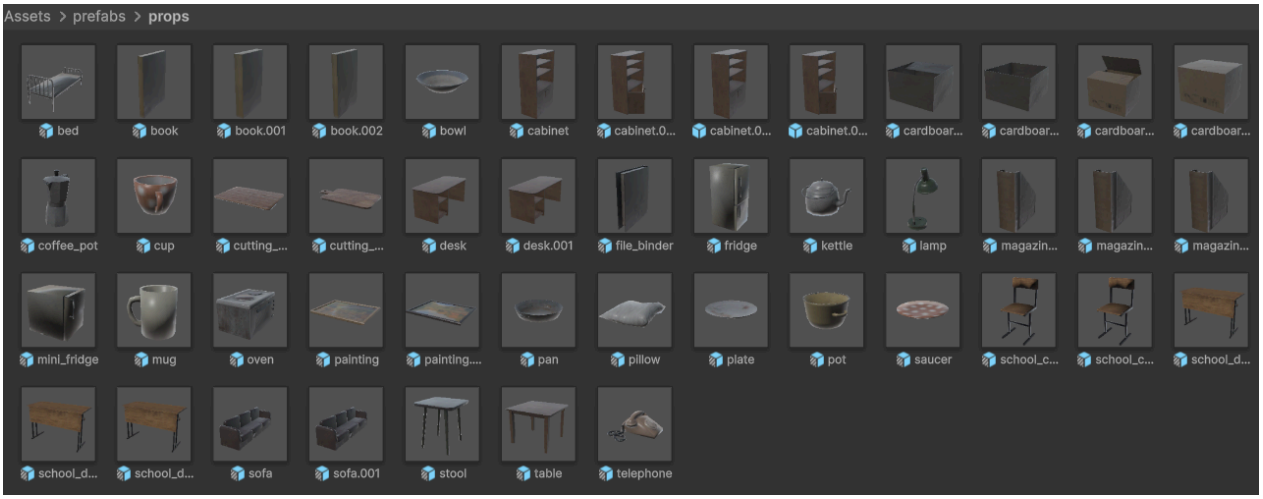


Shelves е обекта, съдържащ графичното представяне на шкафа, на който се нареждат елементите. има само 3 рафта, на които да се поставят обекти, затова не може да има повече от 3 категории. Също така големината му е ограничена, но все пак може да съдържа много слотове за поставяне на тела, тъй като размерите на слотовете и позициите им се задават от XML файла.

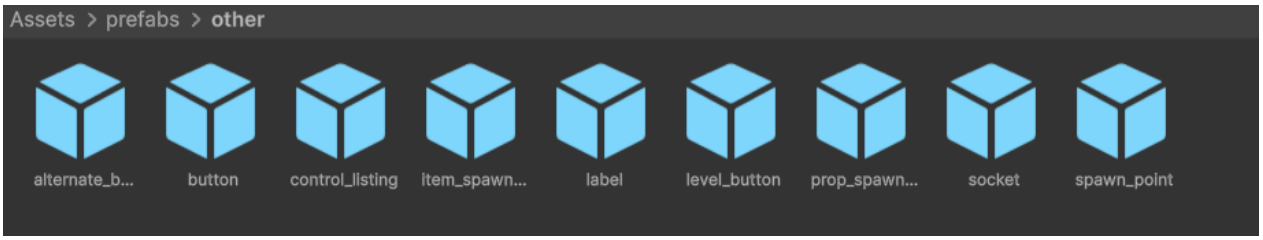
Папка items съдържа всички предефинирани обекти, описващи телата в играта.



Папка props съдържа всички предефинирани обекти, описващи предмети в играта:



Папка other съдържа UI елементи, точките за появяване на тела и предмети, както и слотовете за поставяне на тела в шкафа:



Поведението на обектите в проекта са зададени чрез следните C# скриптове:

- character.cs - отговаря за поведението на обекта играч в проекта. Данните, които управлява и използва, са следните:

```
//constants
const int default_layer = 0;
const int outline_layer = 6;
const int preview_layer = 7;

[Header("configuration")]
[SerializeField] float sensitivity = 8f;
[SerializeField] float reach = 4f;
[SerializeField] float speed = 750;
[SerializeField] float airborne_speed = 250;
[SerializeField] float max_horizontal_velocity_magnitude = 4f;
[SerializeField] float counter_strength = 0.25f;
[SerializeField] float max_slope = 45f;
[SerializeField] float grounded_timeout = 0.125f;
[SerializeField] Material preview_material;
[SerializeField] AudioClip collect_sound;
[SerializeField] AudioClip place_sound;
[SerializeField] AudioClip correct_sound;
[SerializeField] AudioClip incorrect_sound;
[SerializeField] float diminish_multiplier = 1f / 2f;

InputActionMap character_actions;
InputAction move_action;
InputAction look_action;
InputAction drag_action;
InputAction collect_action;
InputAction place_action;
InputAction scroll_belt_action;
InputAction open_backpack_action;
InputActionMap backpack_actions;
InputAction close_backpack_action;

[Header("components")]
[SerializeField] Rigidbody body;
[SerializeField] ConfigurableJoint drag_joint;
[SerializeField] LineRenderer drag_line;
[SerializeField] AudioSource audio_source;
[SerializeField] INVENTORY inventory;
[SerializeField] GameObject backpack_slots_object;
[SerializeField] GameObject cursor_slot_object;

//state
Vector2 move_input = default;
float pitch = 0f;
float grounded_timer = 0f;
bool is_grounded = true;
RaycastHit hit = default;
float drag_distance = 0f;
GameObject outlined_object = null;
GameObject preview_object = null;
GameObject preview_prefab = null;
```

Константите означават слоевете, на които се намират обектите по подразбиране, както и на които трябва да се намират при очертаване с контур и прожектиране в пространството.

Данните под заглавието configuration са параметри за физическата система (speed, airborne_speed, max_horizontal_velocity_magnitude, counter_strength, max_slope, grounded_timeout),

потребителския интерфейс (collect_sound, place_sound, correct_sound, incorrect_sound) и взаимодействието на играча с обектите (sensitivity, reach, preview_material, diminish_multiplier). Те се въвеждат от Unity редактора.

Използва се системата за вход на Unity чрез InputActionMap и InputAction обектите, чиито стойности се вземат от input_actions asset файла.

Данните под заглавието components представляват игрови обекти, които са нужни за функционалностите дърпане на предмет, пускане на звуков сигнал, отваряне на инвентар и предоставянето на визуалния интерфейс. Също така се задават от Unity редактора.

Останалите променливи обозначават състоянието на играта - текущото движение, ъгъла на камерата, състоянието на контакт със земята, информация за колизия между обекти, разстояние до влачения предмет, визуално маркираният в момента обект и временният обект, използван за прожекция на държаното тяло в триизмерното пространство.

```
Unity Message | 0 references
void Awake()
{
    this.character_actions = InputSystem.actions.FindActionMap("character");
    this.move_action = this.character_actions.FindAction("move");
    this.look_action = this.character_actions.FindAction("look");
    this.drag_action = this.character_actions.FindAction("drag");
    this.collect_action = this.character_actions.FindAction("collect");
    this.place_action = this.character_actions.FindAction("place");
    this.scroll_belt_action = this.character_actions.FindAction("scroll_belt");
    this.open_backpack_action = this.character_actions.FindAction("open_backpack");
    this.backpack_actions = InputSystem.actions.FindActionMap("backpack");
    this.close_backpack_action = this.backpack_actions.FindAction("close_backpack");

    this.body.freezeRotation = true;
    this.drag_joint.connectedBody = null;
    this.drag_line.enabled = false;
}

Unity Message | 0 references
void OnEnable()
{
    this.drag_action.started += this.on_drag_start;
    this.drag_action.canceled += this.on_drag_end;
    this.collect_action.performed += this.on_collect;
    this.place_action.performed += this.on_place;
    this.scroll_belt_action.performed += this.on_scroll_belt;
    this.open_backpack_action.performed += this.on_open_backpack;
    this.close_backpack_action.performed += this.on_close_backpack;
    this.character_actions.Enable();
    this.backpack_actions.Disable();
}

Unity Message | 0 references
void OnDisable()
{
    this.drag_action.started -= this.on_drag_start;
    this.drag_action.canceled -= this.on_drag_end;
    this.collect_action.performed -= this.on_collect;
    this.place_action.performed -= this.on_place;
    this.scroll_belt_action.performed -= this.on_scroll_belt;
    this.open_backpack_action.performed -= this.on_open_backpack;
    this.close_backpack_action.performed -= this.on_close_backpack;
    this.character_actions.Disable();
    this.backpack_actions.Disable();
}
```

Функцията Awake се изпълнява при създаване на обекта (играча). В нея се зареждат входните действия от Project Settings, както и се задават стойности по подразбиране на булевите променливи, които отговарят за дърпането на тела.

Функцията OnEnable се изпълнява при активиране на обекта. В нея се регистрират методите, които трябва да се извикват при съответните потребителски действия, тъй като архитектурата е ориентирана към събития (Events). Активира се картата с действия за управление на героя и се деактивира картата с действия за управление на инвентара, за да се гарантира, че играчът не може да управлява едновременно и двете системи.

Функцията OnDisable се изпълнява при деактивиране на игровия обект. Тя премахва регистрираните обработчици на входни събития и деактивира всички входни карти. По този начин се предотвратява изпълнението на действия, когато героят не е активен.

```
Unity Message | 0 references
void Start()
{
    Cursor.lockState = CursorLockMode.Locked;
}

2 references
void on_scroll_belt(InputAction.CallbackContext context)
{
    if (context.ReadValue<Vector2>().y > 0)
    {
        this.inventory.decrement_selected_slot_index();
    }
    else if (context.ReadValue<Vector2>().y < 0)
    {
        this.inventory.increment_selected_slot_index();
    }
}

Unity Message | 0 references
void Update()
{
    this.move_input = this.move_action.ReadValue<Vector2>();
    this.look();

    if (this.drag_joint.connectedBody != null)
    {
        this.drag();
    }

    if (Physics.Raycast(Camera.main.ViewportPointToRay(new Vector3(0.5f, 0.5f, 0f)), out this.hit, this.reach, ~(1 << preview_layer)))
    {
        (ITEM.TYPE item_type, ITEM.STATE _) = this.inventory.selected_slot().peek();
        if (item_type != null)
        {
            if (this.hit.transform.TryGetComponent(out SOCKET socket))
            {
                this.reset_outline();
                this.set_preview(socket.transform.position, socket.transform.rotation, item_type.prefab);
            }
            else if (this.hit.transform.root.TryGetComponent(out BIN _))
            {
                this.set_outline(this.hit.transform.root.gameObject);
                this.reset_preview();
            }
            else if (this.hit.transform.root.TryGetComponent(out ITEM _))
            {
                this.set_outline(this.hit.transform.root.gameObject);
                this.set_preview(this.hit.point, this.camera_facing_rotation(this.hit), item_type.prefab);
            }
            else
            {
                this.reset_outline();
                this.set_preview(this.hit.point, this.camera_facing_rotation(this.hit), item_type.prefab);
            }
        }
        else
        {
            this.reset_preview();
            if (this.hit.transform.root.TryGetComponent(out ITEM _))
            {
                this.set_outline(this.hit.transform.root.gameObject);
            }
            else
            {
                this.reset_outline();
            }
        }
    }
    else
    {
        this.reset_outline();
        this.reset_preview();
    }
}
```

Функцията Start се изпълнява при стартиране на играта. В нея курсорът на мишката се заключва в центъра на екрана, което позволява свободно управление на камерата без визуално разсейване от курсора.

Функцията Update се изпълнява всеки кадър и съдържа основната логика за взаимодействие на играча със средата. В нея се прочитат входните данни за движение и оглеждане, както и се обработва логиката за влачене на предмети. Чрез лъч, излъчен от центъра на камерата, се определя обектът, към който играчът гледа. В зависимост от състоянието на инвентара и типа на засечения обект се управляват визуалните подсказки – очертаване с контур и прожектиране на временен обект за визуализация на поставянето.

Функцията on_scroll_belt се използва за смяна на активния слот в бързата лента на инвентара. Посоката на скролване определя дали изборът се премества напред или назад.

```
1 reference
void look()
{
    float pitch_delta = this.look_action.ReadValue<Vector2>().y * Time.deltaTime * this.sensitivity;
    float yaw_delta = this.look_action.ReadValue<Vector2>().x * Time.deltaTime * this.sensitivity;

    this.pitch = this.pitch - pitch_delta;
    this.pitch = Mathf.Clamp(this.pitch, -90f, 90f);

    Camera.main.transform.localRotation = Quaternion.Euler(this.pitch, 0, 0);
    this.transform.Rotate(0, yaw_delta, 0);
}

2 references
void on_open_backpack(InputAction.CallbackContext context)
{
    this.character_actions.Disable();
    this.backpack_actions.Enable();
    Cursor.lockState = CursorLockMode.None;
    this.backpack_slots_object.SetActive(true);
    this.cursor_slot_object.SetActive(true);
}

2 references
void on_close_backpack(InputAction.CallbackContext context)
{
    this.character_actions.Enable();
    this.backpack_actions.Disable();
    Cursor.lockState = CursorLockMode.Locked;
    this.backpack_slots_object.SetActive(false);
    this.cursor_slot_object.SetActive(false);
}

2 references
void on_drag_start(InputAction.CallbackContext context)
{
    if (this.hit.transform == null) { return; }
    if (this.hit.rigidbody != null)
    {
        this.attach(this.hit);
    }
}

2 references
void on_drag_end(InputAction.CallbackContext context)
{
    if (this.drag_joint.connectedBody != null)
    {
        this.detach();
    }
}

2 references
void on_collect(InputAction.CallbackContext context)
{
    if (this.hit.transform == null) { return; }
    if (this.hit.transform.root.TryGetComponent(out ITEM item))
    {
        if (this.inventory.merge(item.type, item.state) ||
            this.inventory.selected_slot().allot(item.type, item.state) ||
            this.inventory.allot(item.type, item.state))
        {
            if (this.drag_joint.connectedBody == this.hit.rigidbody)
            {
                this.detach();
            }
            Destroy(this.hit.transform.root.gameObject);
            this.audio_source.clip = this.collect_sound;
            this.audio_source.Play();
        }
    }
}
```

Функцията look отговаря за управлението на камерата. Тя преобразува движението на мишката във въртене по хоризонтала и вертикала, като ограничава вертикалния ъгъл, за да се предотврати неестествено обръщане на камерата

Функцията on_open_backpack отваря интерфейса на инвентара. При извикването ѝ управлението на героя се деактивира, курсорът се освобождава и се показват елементите на потребителския интерфейс, свързани с инвентара.

Функцията on_close_backpack затваря интерфейса на инвентара и възстановява управлението на героя. Курсорът отново се заключва, а визуалните елементи на инвентара се скриват.

Функцията on_drag_start се извиква при стартиране на действие за влачене. Ако обектът, към който играчът гледа, има физическо тяло (Rigidbody компонент), той се свързва с героя и може да бъде местен в пространството.

Функцията on_drag_end прекратява влаченето на обекта, като премахва връзката между героя и предмета.

Функцията on_collect реализира събирането на тела от помещението. Ако избраният обект представлява стереометрично тяло, той се добавя към инвентара на играча. След успешно събиране обектът изчезва от сцената и се възпроизвежда звуков ефект.

```
3 references
Quaternion camera_facing_rotation(RaycastHit hit)
{
    //Vector3 camera_direction = Camera.main.transform.position - hit.point;
    //camera_direction.y = 0f;
    Vector3 camera_direction = Camera.main.transform.position - Vector3.up * 1.125f - hit.point;
    Vector3 projection = Vector3.ProjectOnPlane(camera_direction, hit.normal);
    return projection != Vector3.zero ? Quaternion.LookRotation(projection, hit.normal) : Quaternion.FromToRotation(Vector3.up, hit.normal);
}

2 references
void on_place(InputAction.CallbackContext context)
{
    if (this.hit.transform == null) { return; }
    if (this.inventory.selected_slot().count() == 0) { return; }
    if (this.hit.transform.TryGetComponent(out SOCKET socket))
    {
        (ITEM.TYPE item_type, ITEM.STATE item_state) = this.inventory.selected_slot().peek();
        if (item_type.target_id == socket.id)
        {
            this.inventory.set_score(this.inventory.get_score() + item_type.score * item_state.multiplier);
            this.inventory.set_current_tally_count(this.inventory.get_current_tally_count() + 1);
            this.audio_source.clip = this.correct_sound;
            this.audio_source.Play();
            GameObject item_object = Instantiate(item_type.prefab, socket.transform.position, socket.transform.rotation, socket.transform);
            _ = this.inventory.selected_slot().remove();
            Destroy(socket);
        }
        else
        {
            item_state.multiplier = item_state.multiplier * this.diminish_multiplier;
            this.audio_source.clip = this.incorrect_sound;
            this.audio_source.Play();
        }
    }
    else if (this.hit.transform.root.TryGetComponent(out BIN bin))
    {
        (ITEM.TYPE item_type, ITEM.STATE item_state) = this.inventory.selected_slot().peek();
        if (item_type.target_id == bin.id)
        {
            this.inventory.set_score(this.inventory.get_score() + item_type.score * item_state.multiplier);
            this.inventory.set_current_tally_count(this.inventory.get_current_tally_count() + 1);
            this.audio_source.clip = this.correct_sound;
            this.audio_source.Play();
            _ = this.inventory.selected_slot().remove();
        }
        else
        {
            item_state.multiplier = item_state.multiplier * this.diminish_multiplier;
            this.audio_source.clip = this.incorrect_sound;
            this.audio_source.Play();
        }
    }
    else
    {
        (ITEM.TYPE item_type, ITEM.STATE item_state) = this.inventory.selected_slot().remove();
        GameObject item_object = Instantiate(item_type.prefab, this.hit.point, this.camera_facing_rotation(this.hit));
        ITEM item = item_object.AddComponent<ITEM>();
        item.type = item_type;
        item.state = item_state;
        item_object.AddComponent<Rigidbody>().mass = item_type.mass;
        this.audio_source.clip = this.place_sound;
        this.audio_source.Play();
    }
}
```

Функцията camera_facing_rotation изчислява подходяща ориентация на поставяния предмет спрямо повърхността и гледната точка на камерата, така че обектът да бъде разположен естествено в пространството.

Функцията on_place реализира поставянето на предмети от инвентара в средата. Ако играчът опитва да постави предмета на слот в шкафа, на който се подреждат предметите, се извършва проверка дали е поставен на слот с правилното id. При правилно поставяне играчът получава точки и се възпроизвежда положителен звуков ефект, а при грешно множителят за точки се намалява и се възпроизвежда предупредителен звук. Също така се позволява поставяне на обекта на произволно място в пространството, което не намалява точките, който той носи при успешно поставяне.

```
1 reference
void attach(RaycastHit hit)
{
    this.drag_distance = Vector3.Distance(Camera.main.transform.position, hit.point);
    this.drag_joint.transform.position = hit.point;
    this.drag_joint.anchor = Vector3.zero;
    this.drag_joint.connectedBody = hit.rigidbody;
    this.drag_joint.connectedAnchor = hit.rigidbody.transform.InverseTransformPoint(hit.point);
    this.drag_line.enabled = true;
}

1 reference
void drag()
{
    this.drag_joint.transform.position = Camera.main.transform.position + (Camera.main.transform.forward * this.drag_distance);
    this.drag_line.SetPosition(0, this.drag_joint.transform.TransformPoint(this.drag_joint.anchor));
    this.drag_line.SetPosition(1, this.drag_joint.connectedBody.transform.TransformPoint(this.drag_joint.connectedAnchor));
}

2 references
void detach()
{
    this.drag_joint.connectedBody = null;
    this.drag_line.enabled = false;
}

4 references
void set_layer_recursively(GameObject game_object, int layer)
{
    game_object.layer = layer;
    foreach (Transform child_transform in game_object.transform)
    {
        this.set_layer_recursively(child_transform.gameObject, layer);
    }
}

3 references
void set_preview(Vector3 position, Quaternion rotation, GameObject prefab)
{
    if (this.preview_object == null || this.preview_prefab != prefab)
    {
        this.reset_preview();
        this.preview_object = Instantiate(prefab, position, rotation);
        this.preview_prefab = prefab;
        foreach (Collider collider in this.preview_object.GetComponentsInChildren<Collider>())
        {
            collider.isTrigger = true;
        }
        foreach (Rigidbody body in this.preview_object.GetComponentsInChildren<Rigidbody>())
        {
            body.isKinematic = true;
        }
        foreach (var renderer in this.preview_object.GetComponentsInChildren<Renderer>())
        {
            Material[] preview_materials = new Material[renderer.sharedMaterials.Length];
            for (int n = 0; n < preview_materials.Length; n = n + 1)
            {
                preview_materials[n] = preview_material;
            }
            renderer.materials = preview_materials;
            renderer.shadowCastingMode = UnityEngine.Rendering.ShadowCastingMode.Off;
        }
        this.set_layer_recursively(this.preview_object, preview_layer);
    }
    this.preview_object.transform.position = position;
    this.preview_object.transform.rotation = rotation;
}

4 references
void reset_preview()
{
    if (this.preview_object == null) { return; }
    Destroy(this.preview_object);
    this.preview_object = null;
    this.preview_prefab = null;
}
```

Функциите attach, drag и detach реализират механиката за влачене на обекти, като attach свързва избрания предмет с героя, drag обновява позицията му спрямо камерата, а detach прекратява взаимодействието.

Функцията set_layer_recursively задава слоя, на който се намира даден игрови обект. Слоевете се използват, за да се изпълнят различни неща с различните слоеве във функцията Update.

Функциите set_preview и reset_preview управляват визуализацията на временния обект, който показва как би изглеждало поставянето на предмета в пространството, неговата прожекция. Те служат като визуална подсказка за играча.

Функциите set_outline и reset_outline отговарят за визуалното очертаване на обекти, с които играчът може да взаимодейства. Ако курсора е насочен към предмет, той се премества в слоя за очертаване, а ако курсора е преместен от предмета, предметът се премества обратно към default слоя.

```
Unity Message | 0 references
void FixedUpdate()
{
    if (this.grounded_timer > 0f)
    {
        this.grounded_timer -= Time.fixedDeltaTime;
    }
    else
    {
        this.is_grounded = false;
    }

    this.move();
    if (this.is_grounded)
    {
        this.counter_move();
    }
    this.limit_horizontal_velocity();
}

1 reference
void move()
{
    float z = this.move_input.y;
    float x = this.move_input.x;
    this.body.AddForce((this.transform.forward * z + this.transform.right * x) * (this.is_grounded ? this.speed : this.airborne_speed));
}

1 reference
void counter_move()
{
    float yaw = this.transform.eulerAngles.y;
    float velocity_yaw = Mathf.Atan2(this.body.linearVelocity.x, this.body.linearVelocity.z) * Mathf.Rad2Deg;
    float u = Mathf.DeltaAngle(yaw, velocity_yaw);
    float v = 90f - u;
    Vector2 counter_velocity = new Vector2(Mathf.Cos(v * Mathf.Deg2Rad), Mathf.Cos(u * Mathf.Deg2Rad)) * this.body.linearVelocity.magnitude;

    float z = this.move_input.y;
    float x = this.move_input.x;

    if (Mathf.Abs(counter_velocity.x) > 0.5f && Mathf.Abs(x * this.speed) < 0.05f
        || counter_velocity.x < -0.5f && x * this.speed > 0
        || counter_velocity.x > 0.5f && x * this.speed < 0)
    {
        this.body.AddForce(this.transform.right * -counter_velocity.x * this.counter_strength * this.speed);
    }
    if (Mathf.Abs(counter_velocity.y) > 0.5f && Mathf.Abs(z * this.speed) < 0.05f
        || counter_velocity.y < -0.5f && z * this.speed > 0
        || counter_velocity.y > 0.5f && z * this.speed < 0)
    {
        this.body.AddForce(this.transform.forward * -counter_velocity.y * this.counter_strength * this.speed);
    }
}

1 reference
void limit_horizontal_velocity()
{
    Vector3 horizontal_velocity = new Vector3(this.body.linearVelocity.x, 0f, this.body.linearVelocity.z);
    if (horizontal_velocity.magnitude > this.max_horizontal_velocity_magnitude)
    {
        Vector3 limited_velocity = horizontal_velocity.normalized * this.max_horizontal_velocity_magnitude;
        this.body.linearVelocity = new Vector3(limited_velocity.x, this.body.linearVelocity.y, limited_velocity.z);
    }
}

Unity Message | 0 references
void OnCollisionStay(Collision other)
{
    foreach (ContactPoint contact in other.contacts)
    {
        if (Vector3.Angle(Vector3.up, contact.normal) < this.max_slope)
        {
            this.is_grounded = true;
            this.grounded_timer = this.grounded_timeout;
            return;
        }
    }
}
```

Функцията FixedUpdate се изпълнява през фиксиран интервал от време и е предназначена за работа с физичната система на Unity. В нея се управлява състоянието на героя спрямо контакта със земята чрез таймер, който осигурява по-стабилно разпознаване на стъпването върху повърхности. След това се извикват основните функции за движение, противодействие на инерцията и ограничаване на скоростта. По този начин се гарантира, че всички физични изчисления се извършват синхронизирано с физичния енджин и поведението на героя остава предсказуемо и плавно.

Функцията move отговаря за прилагането на сила върху героя в зависимост от входните данни за движение. Посоката на движение се определя спрямо ориентацията на героя в пространството.

Функцията counter_move има за цел да намали нежеланото плъзгане и инерция при спиране или рязка смяна на посоката. Тя анализира текущата хоризонтална скорост на героя и я сравнява с посоката на подадения вход. Когато няма активен вход или когато входът е противоположен на движението, върху героя се прилага продиводействаща сила.

Функцията limit_horizontal_velocity предотвратява натрупването на прекалено голяма хоризонтална скорост. Тя проверява текущата скорост на движение по равнината и при необходимост я ограничава до предварително зададена максимална стойност.

Функцията OnCollisionStay се използва за определяне дали героят е стъпил на повърхност, по която може да се движи. При всяка колизия се анализира ъгълът между нормалата на повърхността и вертикалната ос. Ако ъгълът е в допустимите граници, повърхността се счита за проходима и героят се маркира като заземен.

- session_data.cs - ScriptableObject, в който се пазят данни за текущата игрова сесия. Данните, които се пазят, са име на играча, име на нивото, резултат и време.

```
using UnityEngine;

[CreateAssetMenu(fileName = "session_data", menuName = "session_data")]
Unity Script | 4 references
public class SESSION_DATA : ScriptableObject
{
    public string player_name = null;
    public string level_name = null;
    public float score = 0f;
    public float time = 0f;
}
```

- inventory.cs - отговаря за поведението на инвентара в играта, за други части от потребителския интерфейс, както и за приключването на играта.

```
public class SLOT
{
    //state
    public ITEM.TYPE type;
    Stack<ITEM.STATE> state_stack;

    //events
    public event Action modify_event;
    public event Action select_event;



    3 references
    static public SLOT create()
    {
        SLOT result = new SLOT { };
        result.type = null;
        result.state_stack = new Stack<ITEM.STATE> { };
        return result;
    }



    27 references
    public int count()
    {
        return this.state_stack.Count;
    }

    2 references
    public bool merge(ITEM.TYPE type, ITEM.STATE state)
    {
        if (this.count() > 0 && this.type == type)
        {
            this.state_stack.Push(state);
            this.modify_event?.Invoke();
            return true;
        }
        return false;
    }

    3 references
    public bool allot(ITEM.TYPE type, ITEM.STATE state)
    {
        if (this.count() == 0)
        {
            this.type = type;
            this.state_stack.Push(state);
            this.modify_event?.Invoke();
            return true;
        }
        return false;
    }
```

Класа SLOT е нужен за управлението на слотове в инвентара. В него се пазят типа на държания предмет, стек на състоянията на предмета, събития за променяне и избиране на предмет в инвентара.

Функцията create създава нов празен слот и инициализира вътрешния стек за състояния. Тя осигурява единен начин за създаване на слотове в инвентара.

Функцията count връща броя на предметите, съдържащи се в слота, тъй като един слот може да съдържа няколко на брой предмети от един тип. Използва се за проверка дали слотът е празен или съдържа стек от обекти.

Функцията merge добавя предмет към слот, ако той вече съдържа предмети от същия тип. Така се реализира събирането на еднакви обекти в един слот на инвентара.

Функцията allot поставя предмет в първият празен слот и задава неговия тип. Това е основният начин за добавяне на нов вид предмет в инвентара.

```
3 references
public (ITEM.TYPE type, ITEM.STATE state) remove()
{
    if (this.count() > 0)
    {
        ITEM.TYPE type = this.type;
        ITEM.STATE state = this.state_stack.Pop();
        if (this.count() == 0)
        {
            this.type = null;
        }
        this.modify_event?.Invoke();
        return (type, state);
    }
    return (default, default);
}

3 references
public (ITEM.TYPE type, ITEM.STATE state) peek()
{
    if (this.count() > 0)
    {
        return (this.type, this.state_stack.Peek());
    }
    return (default, default);
}

2 references
public void exchange(SLOT other)
{
    (this.type, other.type) = (other.type, this.type);
    (this.state_stack, other.state_stack) = (other.state_stack, this.state_stack);
    this.modify_event?.Invoke();
    other.modify_event?.Invoke();
}

2 references
public void invoke_select_event()
{
    this.select_event?.Invoke();
}
```

Функцията remove премахва най-горния предмет от слота и го връща за използване в игровия свят. Ако слотът остане празен, неговият тип се нулира.

Функцията peek позволява преглед на текущия предмет в слота, без той да бъде премахнат. Използва се при проверки и визуализация на информация за предмета.

Функцията exchange разменя съдържанието на два слота. Тази функция е основата на логиката за влачене и разместване на предмети в интерфейса на инвентара.

Функцията invoke_select_event задейства събитие за избор на слот. Това позволява потребителският интерфейс да отрази визуално кой слот е активен в момента.

```
[Header("components")]
[SerializeField] SESSION_DATA session_data;

//state
SLOT cursor_slot = SLOT.create();
SLOT[] backpack_slots = null;
SLOT[] belt_slots = null;
int selected_slot_index = 0;
float score = 0f;
int current_tally_count = 0;
int total_tally_count = 0;

//events
public event Action score_event;
public event Action tally_event;



Unity Message | 0 references
void Start()
{
    this.selected_slot().invoke_select_event();
    this.score_event?.Invoke();
    this.tally_event?.Invoke();
}

10 references
public SLOT get_cursor_slot()
{
    return this.cursor_slot;
}

18 references
public SLOT[] get_backpack_slots()
{
    if (this.backpack_slots == null)
    {
        this.backpack_slots = new SLOT[12];
        for (int n = 0; n < this.backpack_slots.Length; n = n + 1)
        {
            this.backpack_slots[n] = SLOT.create();
        }
    }
    return this.backpack_slots;
}
```

Единствената променлива, която се задава от редактора, е session_data, в която се пазят данните за текущата игрова сесия.

Променливите, които определят състоянието на обекта инвентар, са избран слот, слотове в инвентара, слотове в лентата за бърз достъп, индекс на избрания слот, резултат, брой сортирани тела, общ брой тела.

Има и две събития, събитие за добавяне на резултат и за добавяне на сортирано тяло към брояча.

Функцията Start служи за начало на жизненият цикъл на игровия обект и в нея се маркира текущо избраният слот от бързата лента, както и се задействат събитията за обновяване на резултата и броя намерени обекти. По този начин още в началото потребителският интерфейс получава коректна информация за състоянието на играта.

Функцията get_cursor_slot връща слота, който е селектиран в лентата за бърз достъп.

Функцията get_backpack_slots връща всички слотове в инвентарното меню.

```
21 references
public SLOT[] get_belt_slots()
{
    if (this.belt_slots == null)
    {
        this.belt_slots = new SLOT[6];
        for (int n = 0; n < this.belt_slots.Length; n = n + 1)
        {
            this.belt_slots[n] = SLOT.create();
        }
    }
    return this.belt_slots;
}

3 references
public float get_score()
{
    return this.score;
}

2 references
public void set_score(float score)
{
    this.score = score;
    this.score_event?.Invoke();
}

3 references
public int get_current_tally_count()
{
    return this.current_tally_count;
}

2 references
public void set_current_tally_count(int count)
{
    this.current_tally_count = count;
    this.tally_event?.Invoke();

    if (this.current_tally_count == this.total_tally_count)
    {
        this.load_end_screen();
    }
}

1 reference
public void load_end_screen()
{
    this.session_data.score = this.score;
    this.session_data.time = Time.timeSinceLevelLoad;
    SceneManager.LoadScene(GLOBAL.end_screen_scene_number);
}
```

Функцията get_belt_slots връща всички слотове в лентата за бърз достъп. Чрез тези 3 функции се осъществява връзката между инвентарното меню и външния триизмерен свят, като позволява да се достъпят предмети, държани в инвентара на играча.

Функцията get_score предоставя стойността на резултата за визуализация или запис, а set_score го актуализира и задейства събитие, чрез което потребителският интерфейс се обновява автоматично.

Функциите get_current_tally_count и set_current_tally_count отговарят за броя на правилно сортираните обекти в текущото ниво. При всяко успешно поставяне броячът се увеличава, а при достигане на общия брой обекти се счита, че нивото е завършено и се преминава към крайния екран.

Функцията load_end_screen се извиква при успешно завършване на нивото. Тя записва резултата и времето за игра в обекта от тип SESSION_DATA и зарежда менюто за край на играта, където информацията се визуализира на играча.

```
1 reference
public int get_total_tally_count()
{
    return this.total_tally_count;
}

1 reference
public void set_total_tally_count(int count)
{
    this.total_tally_count = count;
    this.tally_event?.Invoke();
}



10 references
public SLOT selected_slot()
{
    return this.get_belt_slots()[this.selected_slot_index];
}

2 references
public void set_selected_slot_index(int index)
{
    this.selected_slot_index = ((index % belt_slots.Length) + belt_slots.Length) % belt_slots.Length;
    this.selected_slot().invoke_select_event();
}

1 reference
public void increment_selected_slot_index()
{
    this.set_selected_slot_index(this.selected_slot_index + 1);
}

1 reference
public void decrement_selected_slot_index()
{
    this.set_selected_slot_index(this.selected_slot_index - 1);
}

1 reference
public bool merge(ITEM.TYPE type, ITEM.STATE state)
{
    for (int n = 0; n < this.get_belt_slots().Length; n = n + 1)
    {
        if (this.get_belt_slots()[n].merge(type, state))
        {
            return true;
        }
    }
    for (int n = 0; n < this.get_backpack_slots().Length; n = n + 1)
    {
        if (this.get_backpack_slots()[n].merge(type, state))
        {
            return true;
        }
    }
    return false;
}
```

Функциите get_total_tally_count и set_total_tally_count определят общия брой обекти, които трябва да бъдат намерени и сортирани в рамките на нивото. Тази стойност се използва като условие за приключване на игровия сценарий.

Функцията selected_slot връща текущо избрания слот от бързата лента. Този слот се използва при всички действия, свързани с поставяне на предмети в света и визуализация на тялото в пространството.

Функцията set_selected_slot_index променя индекса на активния слот в бързата лента, като осигурява циклично преминаване между слотовете. При смяна се задейства събитие, което позволява визуално маркиране на новоизбрания слот.

Функциите increment_selected_slot_index и decrement_selected_slot_index позволяват плавна навигация между слотовете чрез вход от мишката или клавиатурата, като улесняват бързата смяна на активен предмет по време на игра.

Функцията merge се използва при събиране на предмет. Тя проверява дали вече съществува слот със същия тип предмет и, ако е възможно, добавя новото състояние към съществуващия стек. Първо се проверява бързата лента, а след това раницата.

```
1 reference
public bool allot(ITEM.TYPE type, ITEM.STATE state)
{
    for (int n = 0; n < this.get_belt_slots().Length; n = n + 1)
    {
        if (this.get_belt_slots()[n].allot(type, state))
        {
            return true;
        }
    }
    for (int n = 0; n < this.get_backpack_slots().Length; n = n + 1)
    {
        if (this.get_backpack_slots()[n].allot(type, state))
        {
            return true;
        }
    }
    return false;
}
```

Функцията allot се използва, когато няма слот за сливане на предмета. Тя търси първия празен слот, в който да бъде поставен новият предмет, като отново се дава приоритет на бързата лента пред раницата.

- spawner.cs - там се намира логиката за създаването и разположението на триизмерните тела и предмети в стаята, както и прочитането на нужните данни от XML файлове

```
[XmlRoot("configuration")]
5 references
public class XML_CONFIGURATION
{
    [XmlArray("item_pools")]
    [XmlArrayItem("item_pool")]
    public List<XML_ITEM_POOL> item_pools = new List<XML_ITEM_POOL>();

    [XmlArray("prop_pools")]
    [XmlArrayItem("prop_pool")]
    public List<XML_PROP_POOL> prop_pools = new List<XML_PROP_POOL>();

    [XmlArray("item_spawn_points")]
    [XmlArrayItem("item_spawn_point")]
    public List<XML_ITEM_SPAWN_POINT> item_spawn_points = new List<XML_ITEM_SPAWN_POINT>();

    [XmlArray("prop_spawn_points")]
    [XmlArrayItem("prop_spawn_point")]
    public List<XML_PROP_SPAWN_POINT> prop_spawn_points = new List<XML_PROP_SPAWN_POINT>();

    [XmlArray("props")]
    [XmlArrayItem("prop")]
    public List<XML_STATIC_PROP> props = new List<XML_STATIC_PROP>();
}

3 references
public class XML_ITEM_SPAWN_POINT
{
    [XmlAttribute("item_pool_id")]
    public int item_pool_id;

    [XmlElement("position")]
    public Vector3 position;

    [XmlElement("rotation")]
    public Vector3 rotation;
}

3 references
public class XML_PROP_SPAWN_POINT
{
    [XmlAttribute("prop_pool_id")]
    public int prop_pool_id;

    [XmlElement("position")]
    public Vector3 position;

    [XmlElement("rotation")]
    public Vector3 rotation;
}

3 references
public class XML_STATIC_PROP
{
    [XmlElement("prefab")]
    public XML_PREFAB prefab;

    [XmlElement("position")]
    public Vector3 position;

    [XmlElement("rotation")]
    public Vector3 rotation;
```

.

Тези класове, както и последващите ги, служат за работа с данните от XML файловете при прочитането на данните от тях. Представляват класове със същите елементи, каквито има и в структурата на XML файловете, описана по-горе.

В началото на изпълнението, във функцията Start, се изгражда пътят до XML файла на текущото ниво, като се използва името на нивото, съхранено в SESSION_DATA. Файлът се отваря и десериализира чрез XmlSerializer, като по този начин съдържанието му се преобразува в обект от тип XML_CONFIGURATION. От този момент нататък всички параметри на нивото се управляват чрез данните, описани във външния документ.

След зареждането на конфигурацията първо се създават обектите, които представляват точки за създаване на тела и предмети. За всяка такава точка се създава празен GameObject в сцената, задават се неговите позиция и ротация и се добавя съответният компонент, който съдържа идентификатора на пула, от който ще се избира обект. По този начин XML документът определя логическите места в помещението, без сам да съдържа реални Unity обекти.

След това се създават статичните предмети, описани в конфигурацията. За всеки проп се намира съответният prefab по име и се инстанцира в сцената. Ако към него е дефиниран контейнер за сортиране, се добавя компонент, който пази неговия идентификатор. Ако са описани сокети, за всеки от тях се създава дъщерен обект с колайдер и уникален идентификатор. Тези сокети представляват зоните, в които играчът трябва да предметиправилните предмети. Ако са зададени текстови етикети, се създават обекти с TextMeshPro компонент, на които се задават текст, размер и подравняване. Така чрез XML се определя не само геометрията на помещението, но и неговото информационно съдържание.

След изграждането на статичната част на нивото се преминава към динамичното създаване на пропове и предмети чрез функциите spawn_prop и spawn_item. Функцията spawn_prop избира случаен елемент от съответния пул въз основа на тегла за вероятност и го инстанцира в позицията на точката за създаване. Ако създаденият обект съдържа вложени точки за създаване, функцията се извиква рекурсивно, което позволява изграждането на йерархични структури от обекти.

Функцията spawn_item работи по сходен начин, но създава телата, които играчът може да събира. Тя проверява дали е достигнат максималният брой за съответния пул, избира елемент според теглата, създава неговия prefab и добавя компонент ITEM. При първо създаване на даден тип предмет се създава и обект от тип ITEM.TYPE, който съдържа спрайт, описание, маса, стойност на точките и идентификатор на целевия контейнер. Така се осигурява връзка между визуалното представяне, физическите характеристики и логиката за сортиране.

Функциите sample_item_pool и sample_prop_pool реализират механизъм за случаен избор с тежести, което позволява различните обекти да се появяват с различна вероятност. Това прави разположението на предметите частично случайно и увеличава разнообразието при всяко стартиране на нивото.

Функцията load_sprite зарежда 2D изображение от файл в постоянната директория на приложението и го преобразува в Sprite, който впоследствие се използва в инвентара.

- От класа SPAWNER зависят item_spawner.cs и prop_spawner.cs, които съдържат съответните класове, съдържащи единствено id-то на пула, от който да се избират предметите или телата.
- Файлът item.cs съдържа класа ITEM, който съдържа информация за тялото, което играчът може да събира и поставя. Той съдържа подклас тип с полетата prefab, спрайт, описание, маса, id на таргетния бин (мястото, където трябва да се поставят еднотипните тела), както и подклас състояние, което съдържа полето множител, с който се намалят точките, които тялото дава на играча при правилно поставяне.
- Файловете bin.cs и socket.cs съдържат само id, което се отнася за бина или за слота в бина. Използват се при детекция дали играчът поставя тяло на правилното място.
- Файлът global.cs съдържа глобални променливи, свързани с имената на нивата и индексите на сцените.
- В папка overlays се намират класове, свързани с потребителския интефейс в игровата сцена:
    - backpack_slot.cs - реализира визуалното и интерактивното представяне на един слот от инвентара в потребителския интерфейс, свързва логическите данни от съответния SLOT в инвентара с икона на елемента, брой предмети и визуално описание
    - belt_slot.cs - подобно на backpack_slot, но се отнася за слота в лентата за бърз достъп.
    - В cursor_slot.cs се визуализира съдържанието на предмета, който играчът държи при преместване в селектирания слот на лентата за бърз достъп.
    - Файловете score.cs, tally.cs, time.cs отговарят съответно за визуализирането и ъпдейтването на интерфейсните елементи, които отговарят за показването на съответно резултата на играча, броя на сортираните тела от всички налични, както и времето, което е минало от началото нивото до всеки един момент.
    - Файлът tooltip.cs отговаря за визуализирането на етикет, показващ името на предмета, щом курсора е насочен към него в менюто на инвентара.
- В папка screens се намират класове, свързани с менютата на играта:
    - Класът MAIN_MENU в main_menu.cs реализира логиката на главното меню на играта. Той управлява въвеждането на име на играча, визуализацията на наличните нива, проверката за отключване на нива чрез предварителни условия, както и показването на контролите:

      Функцията Start се изпълнява при стартиране на сцената. В нея се регистрира слушател към полето за въвеждане на име, така че всяка промяна да преминава през валидиране. След това създава визуалния списък с контроли и бутоните за нивата.

Функцията create_control_listings извлича действията от Input System картата „character“ и за всяко действие създава UI елемент, който показва неговото име и съответния клавиш в менюто за контроли.

Във функцията create_level_buttons се прочитат файловете от директорията с нива. За всеки валиден файл се създава бутон и задава неговия текст според името на файла. При натискане на бутона се извиква load_level, която зарежда избраното ниво.

Функцията init_level_buttons проверява за всяко ниво дали има предварително условие за отключване. Ако такова съществува и играчът не го покрива, бутонът се скрива.

Функцията get_prerequisite прочита XML файла на нивото и извлича стойността на елемент, показващ условието за отключване на ниво. Методът player_meets_prerequisite проверява в leaderboard файла дали текущият играч е покрил изискването, като търси съответстващ запис.

Функцията validate_name проверява въведеното име за валидност. Името не може да бъде празно и не може да съдържа запетая или нов ред, тъй като тези символи се използват при запис във файла с резултати. Ако името вече съществува, се показва предупредително съобщение, но не се блокира използването му. Методът name_is_in_use извършва проверка в leaderboard файла.

Функцията save_name записва валидното име в SESSION_DATA, инициализира бутоните за нива според прогреса и превключва от екрана за въвеждане на име към екрана за избор на ниво, а функцията load_level записва избраното име на ниво в SESSION_DATA и зарежда сцената на нивото чрез SceneManager. Методът quit прекратява изпълнението на приложението.

- Във файла end_screen.cs се съдържа класът END_SCREEN, в който се съдържа логиката за визуализирането на SESSION_DATA в края на изиграно ниво.
- Във файла button.cs се съдържа реализацията на бутони, които се използват в менютата на играта.
- Във файла spinner.cs се съдържа логиката за постигането на въртящо случайно тригонометрично тяло като фон на началния екран.

3.4 Дизайн на нивата на играта

Нивата на играта се задават от XML файлове, затова могат да бъдат конфигурирани външно. В тази версия на играта съществуват 4 нива с различни трудности, като трудността се определя от сложността на учебния материал, отразен в тях. В по-трудните нива телата не са назовани по име, за да се увеличи трудността. Примерните нива представляват сценарии, в които играчът трябва да сортира телата в тези категории:

- Ротационни и многостенни тела;

- Изпъкнали и вдлъбнати тела;
- Пирамиди, антипризми, призми;
- Платонови, върхово-транзитивни и стенно-транзитивни тела.

4 Използване на изкуствен интелект

В рамките на проекта изкуствения интелект е използван основно като средство, чрез което да се запознаем с нови за нас концепции при създаването на играта. Тъй като и двамата имахме минимални знания за Unity преди да започнем проекта, често използвахме ИИ с цел да разберем как точно работи някоя структура в езика, тъй като синтезираната информация, с която той отговаря, е по-лесно смилаема, отколкото би била същата информация, описана във формалната документация. Тоест за този проект не сме използвали ИИ като генератор за код, а като учител, от който да разберем повече за избраната технология. Заради това не ни се е налагало да задаваме контекста на решението си.

Промпт, използван при такъв случай: In unity how do i make a editor script that exports the selected game objects position and rotation as a formatted string and copies it to my clipboard

Отговор от GPT-5.2:

"In Unity, you can do this by creating a **custom editor script** that:

1. Gets the currently selected GameObject(s)
2. Reads their transform.position and transform.rotation
3. Formats the data into a string
4. Copies it to the clipboard using EditorGUIUtility.systemCopyBuffer

Step 1 — Create an Editor Folder

Create this folder if you don't already have one:

Assets/Editor/

Then create a C# script inside it, for example:

ExportTransformTool.cs

Step 2 — Editor Script

Here's a complete working example:

```
using UnityEngine;
using UnityEditor;
using System.Text;

public static class ExportTransformTool
{
    [MenuItem("Tools/Export Selected Transforms %#e")] // Ctrl+Shift+E
    private static void ExportSelectedTransforms()
    {
        GameObject[] selectedObjects = Selection.gameObjects;

        if (selectedObjects.Length == 0)
        {
            Debug.LogWarning("No GameObjects selected.");
            return;
        }

        StringBuilder sb = new StringBuilder();

        foreach (GameObject obj in selectedObjects)
        {
            Transform t = obj.transform;

            Vector3 pos = t.position;
            Vector3 rot = t.eulerAngles;
```

```
            sb.AppendLine($"{obj.name}:");
            sb.AppendLine($"Position: ({pos.x:F3}f, {pos.y:F3}f, {pos.z:F3}f)");
            sb.AppendLine($"Rotation: ({rot.x:F3}f, {rot.y:F3}f, {rot.z:F3}f)");
            sb.AppendLine();
        }

        string result = sb.ToString();

        EditorGUIUtility.systemCopyBuffer = result;

        Debug.Log("Transform data copied to clipboard:\n" + result);
    }
}
```

How It Works

- `Selection.gameObjects` → gets all selected objects
- `t.position` → world position
- `t.eulerAngles` → rotation in degrees
- `EditorGUIUtility.systemCopyBuffer` → copies text to clipboard
- `MenuItem("Tools/...")` → adds a menu option in Unity

You'll now find it under:

`Tools → Export Selected Transforms`

Shortcut:

`Ctrl + Shift + E`

Example Output

Cube:

Position: (1.000f, 2.000f, 3.000f)

Rotation: (0.000f, 90.000f, 0.000f)

Enemy:

Position: (5.234f, 0.000f, -2.111f)

Rotation: (0.000f, 180.000f, 0.000f)

Optional Improvements

Export Local Instead of World

Replace:

t.position

t.eulerAngles

With:

t.localPosition

t.localEulerAngles

Export As Code-Ready Format

If you want something like:

new Vector3(1f, 2f, 3f)

Replace the formatting with:

```
sb.AppendLine(
    $"new Vector3({pos.x:F3}f, {pos.y:F3}f, {pos.z:F3}f), " +
    $"Quaternion.Euler({rot.x:F3}f, {rot.y:F3}f, {rot.z:F3}f)"
```

```
);
```

Също така сме използвали ИИ в началната фаза на проекта, когато избирахме тема на образователната игра. Беше полезен с това, че даде няколко идеи за тематика на играта, макар че не се спряхме на никой от тях в крайна сметка. ИИ помогна и при избирането на стереометрични тела, които да използваме в учебните сценарии, както и при подреждането им в категории.

Използвани модели: GPT-5.2, Grok Expert, Gemini 3

5 Тестване

Цялото тестване на играта се състоя в Unity редактора, като беше по-скоро итеративно - след всяка нова итерация на проекта, новите добавки бяха тествани, преди да бъдат въведени към цялостното решение. В края на всяка нова итерация по проекта правихме тестове на текущата итерация, като най-важно беше дали играта е цялостна и годен за игра. След завършването на играта, беше тествана по-усилено от нас, а и също беше показвана на трети лица с цел плей тестинг. Въпреки че надали всички бъгове са открити на този етап, играта е в стабилно състояние и може да бъде играна, геймплей цикъла е цялостен.

6 Заключение и възможно бъдещо развитие

В заключение, работата по проекта беше изключително полезна за нас, тъй като разширихме знанията си по проектиране на игри, както и за двигателя Unity. Използваните методи на XML при зареждането на нивата в играта е удобно, тъй като прави промените на малки неща в игровия процес лесно, без да има нужда да се променя кода. По този начин играта може да бъде използвана за различни учебни сценарии, като лицето, използващо я в учебния си план, може да променя сценариите, без да има нужда от достъп до сорс кода. Също така могат да се създават нови нива с различни разположения и цели, без да се използва Unity редактора. Ограничението се състои в това, че XML файловете все пак имат строга структура, както и че през тях не може да се задават 3D модели на предметите. Също така физическото взаимодействие чрез Rigidbody компоненти, макар и реалистично, може да доведе до непредвидимо поведение при по-сложни сцени.

Като насоки за бъдещо развитие, може в играта да се добавят нови нива, различни режими на игра (например различни игрови сценарии, засягащи други учебни дисциплини),

система за подсказки, различни помещения, както и графичен редактор за нива, който да ги конвертира в XML файлове. В крайна сметка има голям брой неща, които могат да бъдат добавени към играта, заради простотата на нейния геймплей.

7 Разпределение на работата

По време на проекта работата не беше разпределена методично, като предпочетохме да работим по каквото се наложи да бъде свършено в текущия момент. За някои части на проекта работата беше съвместна, но като цяло беше разпределена така:

- Борил: работа по XML частта, създаване на механизма за интеракция с тела и предмети, разработка на шкафа за поставяне на телата, намиране на ротационни тела за играта, създаване на механизма за разполагане на предмети и тела на случайни места в стаята.
- Виктор: създаване на инвентарната система, намиране на триизмерни обекти за предмети в играта, разработка на началния и крайния екран, записване на потребителската информация, документиране на процеса.